

dQwen3.5: Hybrid-Attention Diffusion Language Models

Anton Xue Litu Rout Aditya Akella
Adam Klivans Sujay Sanghavi Sanjay Shakkottai

University of Texas at Austin

{anton.xue, litu.rout, sanjay.shakkottai}@utexas.edu,
{akella, klivans}@cs.utexas.edu, sanghavi@mail.utexas.edu

Abstract

Adapting a pretrained autoregressive (AR) model is a cost-efficient route to a diffusion language model (DLM). While nearly all such adaptations start from a full-attention transformer, AR modeling has shifted toward hybrid architectures that interleave attention and RNN layers. This creates an obstacle for adaptation: unlike attention, RNNs are structurally causal and nontrivial to bidirectionalize. Despite this mismatch, we investigate whether such backbones can become effective DLMs by adapting Qwen3.5 at 0.8B, 2B, 4B, and 9B scales, yielding the dQwen3.5 family. We find that hybrid backbones can be efficient starting points for adaptation: against a full-attention control, the hybrid reaches a given training loss in about half the tokens. Across scales, dQwen3.5 resembles full-attention DLMs in any-order decoding behavior and performs strongly under parallel decoding.

 huggingface.co/UT-IFML/dQwen3.5-9B-Base

1 Introduction

Diffusion language models (DLMs) enable parallel and any-order generation by iteratively unmasking positions rather than strictly decoding left to right [Austin et al., 2021a, Sahoo et al., 2024, Shi et al., 2024]. While recent releases are increasingly competitive with autoregressive (AR) models [Nie et al., 2025b, Ye et al., 2025], training a DLM from scratch remains expensive: reaching parity with a similarly sized AR model can require an order of magnitude more tokens [Nie et al., 2025a]. AR-to-DLM adaptation has thus emerged as a cheaper alternative for training a DLM: one initializes from a pretrained AR model, makes its causal attention layers bidirectional, and continues training with a diffusion objective [Cheng et al., 2026, Gong et al., 2025, Ye et al., 2025]. While nearly every such adaptation to date has started from a full-attention transformer, AR modeling has meanwhile moved toward *hybrid* architectures that interleave attention with recurrent (RNN) layers [Lieber et al., 2024, MiniMax, 2025, Qwen Team, 2026]. Unlike attention, RNN layers are structurally causal and non-trivial to bidirectionalize, which raises a question: can one nevertheless adapt hybrid backbones into performant DLMs?

At first glance, this structural causality seems poorly matched for any-order generation, one of the defining capabilities of DLMs. Specifically, while an attention causal mask can be easily modified for bidirectionality, the same is not true for the RNN layers that make up most of a hybrid model’s

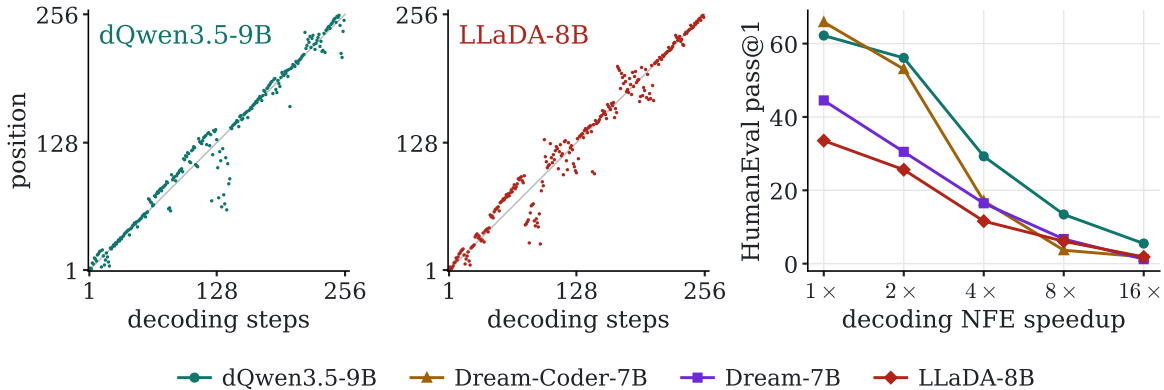


Figure 1: **Hybrid backbones make strong DLMs.** Despite causal RNN layers, dQwen3.5-9B supports both any-order and parallel decoding. *Left, Center:* on HumanEval/27, dQwen3.5-9B and the full-attention, from-scratch LLaDA-8B globally decode left-to-right, but locally decode out-of-order (Section 4.3); each dot marks a position’s decode step. *Right:* at higher NFE speedups, dQwen3.5-9B retains more HumanEval performance than the comparable-trunk DLMs we evaluate.

layers. Thus, although each position can, in principle, obtain future context through bidirectional attention layers, the architecture maintains a strong causal bias that conflicts with any-order generation. Fortunately, this mismatch may be overstated: natural language is produced and read from left to right, and the information useful for predicting a token is concentrated in the tokens immediately preceding it [Khandelwal et al., 2018, O’Connor and Andreas, 2021, Sun et al., 2021]. In fact, DLMs with full-attention backbones exhibit a global left-to-right decoding trend, even for models like LLaDA that are trained from scratch [Gong et al., 2026]. Together, these observations suggest that the RNN’s causal bias may, in fact, be complementary to bidirectionalized attention layers (see also Section 5 for additional discussion).

Motivated by this view, we adapt Qwen3.5 at 0.8B, 2B, 4B, and 9B scales into hybrid-attention DLMs, which we call dQwen3.5. As suggested above, we bidirectionalize only the attention layers, leave the RNN layers causal, and train all four sizes under a shared adaptation recipe. Additionally, we adapt Qwen3-1.7B [Yang et al., 2025a] as a full-attention control under the same recipe, where notably its trunk (non-embedding, non-head) parameter size closely matches that of Qwen3.5-2B. With this setup, we then study whether adapted hybrid-attention AR models make for good DLMs: whether they exhibit the any-order and parallel decoding expected of DLMs, and whether such backbones provide a competitive starting point for AR-to-DLM adaptation.

Our experiments indicate that hybrid-attention adaptations support both any-order and parallel decoding. As previewed in Figure 1, dQwen3.5-9B resembles the full-attention, from-scratch LLaDA-8B: generation trends globally left to right while nearby positions are often decoded out of order. We quantify this distinction with two decode-order metrics: *local AR-ness* measures left-to-right ordering among adjacent positions (e.g., k and $k + 1$), while *global AR-ness* measures it over all pairs (e.g., $i < j$); random order and strict AR decoding score 0.5 and 1.0, respectively (Section 4.3). Across all four dQwen3.5 scales, local AR-ness is about 0.64, within the 0.58–0.66 range of other evaluated DLMs, while global AR-ness is much higher for all models at roughly 0.88–0.97. For parallel decoding, we measure accuracy against decoding speedup (e.g., $2\times$ decodes

two tokens per step; Section 4.4). As shown in Figure 1, dQwen3.5-9B leads the comparable-trunk DLMs we evaluate on HumanEval at every speedup beyond $1\times$.

Beyond their DLM-like decoding behavior, hybrid backbones can also adapt quickly. Against our trunk-matched full-attention control, dQwen3.5-2B reaches a given training loss in roughly half the tokens. At this same $\approx 1.4\text{B}$ trunk scale, dQwen3.5-2B after 50B tokens also outperforms CoDA [Chen et al., 2025] (200B tokens) on 6/7 benchmarks. At the larger 6.5–7B trunk scale, dQwen3.5-9B after 50B tokens attains the highest score on 4/7 benchmarks against Dream-7B [Ye et al., 2025] (580B tokens), Dream-Coder-7B [Xie et al., 2025] (322B tokens), and LLaDA-8B [Nie et al., 2025b] (2.3T tokens). Interestingly, longer adaptation is not uniformly better: moving from 50B to 100B improves 5/7 benchmarks at 0.8B, but only 1/7 at 9B.

We give the full adaptation recipe in Section 3 and evaluation in Section 4. We further release the base dQwen3.5 models at 0.8B, 2B, 4B, and 9B scales, together with the full-attention control. With a shared architecture family and adaptation recipe, these releases provide a useful reference for studying hybrid AR-to-DLM adaptation across scale.

1.1 Related work

Diffusion language models. Discrete diffusion models generate text directly over tokens, rather than in the continuous embedding space of Diffusion-LM [Li et al., 2022]. Notable instances include D3PM [Austin et al., 2021a], MD4 [Shi et al., 2024], MDLM [Sahoo et al., 2024], and SEDD [Lou et al., 2024], followed by scaling studies [Nie et al., 2025a]. Open releases now range from LLaDA-8B [Nie et al., 2025b] and its successor iLLaDA-8B [Nie et al., 2026] to the 100B-scale LLaDA 2.0 [Bie et al., 2025], while closed-weight models include Mercury [Inception Labs et al., 2025]. Although these models demonstrate that DLMs can scale to strong language models, doing so from scratch remains substantially more expensive than training comparable AR models, motivating adaptation from pretrained AR checkpoints.

AR-to-DLM adaptation. Rather than pretraining from scratch, a DLM can be obtained by adapting a pretrained AR model. Representative examples include DiffuLLaMA [Gong et al., 2025] (65B tokens), Dream-7B [Ye et al., 2025] (580B), Dream-Coder-7B [Xie et al., 2025] (322B), Diffu-Coder [Gong et al., 2026] (130B), CoDA [Chen et al., 2025] (200B), Efficient-DLM [Fu et al., 2026b] (300–500B), and Nemotron-Labs-Diffusion [Fu et al., 2026a] (300B), compared with roughly 2T–12T tokens for from-scratch DLM pretraining [Nie et al., 2025b, 2026]. Most such adaptations use full-attention backbones. SDAR [Cheng et al., 2026] studies adaptation across the Qwen3 family, but releases only post-trained weights, complicating base-model comparison. Closest to ours, FLARE [Zhu et al., 2026] adapts hybrid Qwen3.5 backbones, but does not directly compare hybrid and full-attention backbones under adaptation.

Causal structure in language modeling. Natural language itself exhibits a strong left-to-right bias: token predictions depend heavily on nearby prefix context, while distant context contributes more selectively [Khandelwal et al., 2018, O’Connor and Andreas, 2021, Sun et al., 2021]. Modern hybrid AR models encode this bias architecturally by interleaving attention with causal recurrent layers [Lieber et al., 2024, MiniMax, 2025, Qwen Team, 2026, Yang et al., 2025b]. DLMs exhibit a similar tendency at inference time: even when decode order is unconstrained, generation progresses broadly left to right, including for models such as LLaDA that were trained from scratch [Gong et al.,



Figure 2: **Full-canvas and block decoding.** Gray slots denote masks, while the prompt (“The quick”) remains fixed. Both schemes begin from the same masked canvas and decode 2 tokens per step. Full-canvas decoding (left) may unmask anywhere, whereas block decoding (right) restricts unmasking to the active block (dashed outline), advancing to the next block once it is complete.

2026]. Causality can also be imposed explicitly through strategies such as block decoding [Arriola et al., 2025]. Our work instead asks whether substantial causal structure can remain in the backbone itself while preserving the any-order and parallel decoding behavior of a DLM.

2 Masked diffusion language models

A masked diffusion language model (DLM) generates text by iteratively decoding (unmasking) masked positions rather than generating strictly from left to right [Austin et al., 2021a, Sahoo et al., 2024, Shi et al., 2024]. During training, positions are randomly masked at varying rates and the model predicts the conditional marginal distribution (equivalently, the logits) corresponding to the original tokens at the masked positions, allowing each prediction to use context from both its left and right. At generation time, prompt tokens remain fixed while the masked positions form a *canvas*; at each step, the model predicts token logits for the masked positions and a subset is unmasked (sampled from the logits) until the canvas is fully decoded. Which positions to decode are commonly selected by confidence (top-1 token probability), either through top- k or threshold-based schemes [Nie et al., 2025b, Ye et al., 2025].

In Figure 2, an 8-position canvas unmask 2 tokens per step and is therefore fully decoded in 4 steps. *Full-canvas decoding* may decode positions anywhere on the canvas, whereas *block decoding* partitions it into fixed-size blocks—of size 4 in the figure—and completes each block before advancing, yielding a semi-autoregressive trajectory that is left-to-right across blocks but any-order within the active block [Arriola et al., 2025]. We study both in Section 4.4 and Section C.5, and give the formal diffusion process and training objective in Section A.

2.1 Recurrent and hybrid language-model backbones

A growing class of language models interleaves self-attention with recurrent neural network (RNN) layers, often making the recurrent layers the majority of the architecture stack [Lieber et al., 2024, MiniMax, 2025, Qwen Team, 2026, Yang et al., 2025b]. Gated DeltaNets (GDNs) are one such modern RNN: they scan the sequence left to right while compressing the prefix into a fixed-size recurrent state [Yang et al., 2025b]. Qwen3.5 uses GDNs as its main sequence-processing layer, with three GDN layers for every standard attention layer [Qwen Team, 2026] (see Figure 3). Consequently,

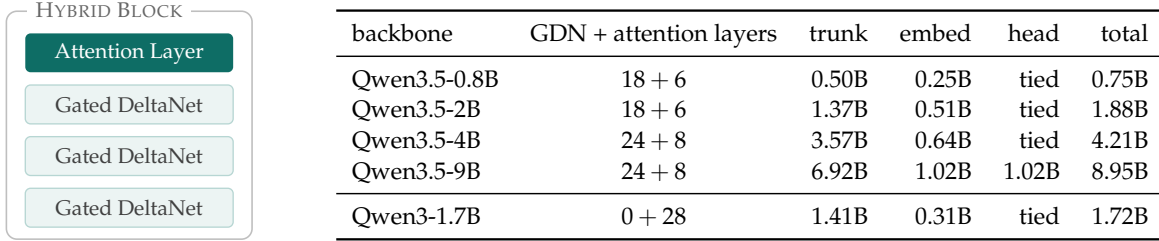


Figure 3: **Bidirectionalizing a hybrid model.** Each hybrid block applies three GDNs before a single attention layer; we bidirectionalize only the attention and leave the GDNs causal. The table reports layer and parameter counts across scales, including the Qwen3-1.7B full-attention control.

causality in a GDN is built into its recurrence rather than imposed by an attention mask, and so bidirectionalization would require non-trivial architectural modifications.

3 Adaptation recipe

Our adaptation follows standard AR-to-DLM practice wherever possible; the main difference is that only Qwen3.5’s attention layers are made bidirectional, while its recurrent layers remain causal. We make no optimality claims about individual recipe choices and report the exact configuration used in our experiments.

3.1 Architecture modifications

We adapt Qwen3.5-0.8B, Qwen3.5-2B, Qwen3.5-4B, and Qwen3.5-9B. These checkpoints include vision components and an auxiliary multi-token-prediction head; we discard both and adapt only the language-model backbone, which is also why our parameter counts differ from those reported on the model cards. The language-model backbone consists of stacks of hybrid blocks (Figure 3); we modify only their attention layers and otherwise leave the stack structurally unchanged. For comparison, we also adapt Qwen3-1.7B as a full-attention control, whose 1.41B-parameter trunk closely matches the 1.37B-parameter trunk of Qwen3.5-2B.

Bidirectionalization. Across all four Qwen3.5 sizes, attention makes up 25% of the sequence-processing stack (6/24 layers for the two smaller models and 8/32 for the two larger ones); we make these layers bidirectional by disabling their causal masks. The remaining Gated DeltaNet layers are causal by construction (Section 2.1) and are left untouched. Thus, only a minority of the stack becomes bidirectional, while the recurrent majority retains the causal structure studied in this work. For the full-attention control, every sequence-processing layer is attention and is therefore bidirectionalized. The remaining modifications follow standard AR-to-DLM adaptation practice.

Token shifting. We prepend a BOS token and shift the readout by one position, so that the hidden state at position k predicts the token at position $k + 1$ when that target position is masked (Figure 4). This preserves the next-token alignment of the pretrained AR model while allowing the input itself to contain masked tokens. Token shifting is standard across the AR-to-DLM literature and is known to help adaptation [Gong et al., 2025, Xie et al., 2025, Ye et al., 2025].

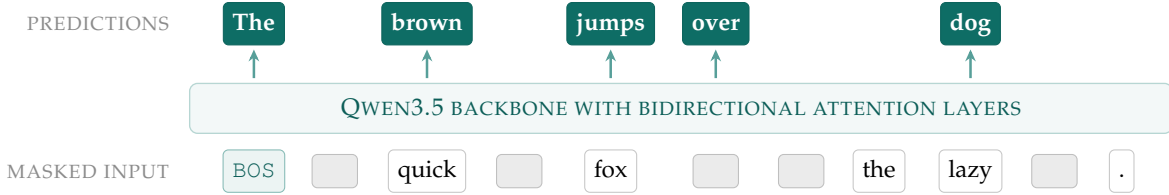


Figure 4: **Token shifting preserves the AR readout alignment.** The hidden state at position k predicts the token at position $k + 1$, matching the readout used during AR pretraining while allowing bidirectional attention to incorporate future context. This alignment is known to improve AR-to-DLM adaptation [Gong et al., 2025, Ye et al., 2025].

Special tokens. We additionally need three special tokens for diffusion language modeling with token shifting: a mask token for the diffusion objective, a padding token the model emits to halt generation, and a leading BOS for the shifted readout. We repurpose unused ids rather than resizing the embedding table: mask is Qwen3.5’s `<|fim_middle|>`, padding its `<|endoftext|>`, and BOS its `<tts_text_bos>`.

3.2 Training data and configuration

We give an overview here; the complete training configuration is in Section B.1, with our learning-rate and mixture explorations summarized in Section B.3.

Mixture. We train on a fixed mixture of 14 subsets drawn from NVIDIA’s Nemotron pretraining releases¹ [Karimi Mahabadi et al., 2026, NVIDIA, 2025a,b, Su et al., 2025]: 50% code, 35% general text, and 15% mathematics by token share; Section B.1 lists the subsets and their weights. Sequences are packed to 4096 tokens, and a small fraction is randomly truncated so that not every example fills the context. Before applying diffusion masking, each 512-sequence batch contains about 1.99M content (non-mask, non-pad) tokens on average after truncation and padding.

Training. We train a time-reweighted form of the standard masked diffusion objective [Austin et al., 2021a, Sahoo et al., 2024, Shi et al., 2024], following Shi and Titsias [2025], defined in Section A. Across all sizes, we use the same optimizer and warmup–stable–decay schedule, with only the learning rate varying by model size (3, 2, 2, 1×10^{-5} for 0.8B/2B/4B/9B). The global batch is 512 sequences $\times$ 4096 tokens. For each size, we retain checkpoints at two budgets: 25k steps (1k / 19k / 5k, 50B content tokens) and 50k steps (1k / 44k / 5k, 100B content tokens). These two budgets let us study how quickly useful diffusion capabilities emerge and whether further adaptation continues to help. We checkpoint every 1,000 steps and release the weight average of the last five checkpoints: steps 21,000–25,000 at the shorter budget and 46,000–50,000 at the longer.

3.3 Training dynamics

The training curves are smooth across sizes, but loss should be interpreted carefully. In particular, the 4B and 9B loss curves in Figure 9 nearly overlap despite separating clearly on downstream benchmarks (Section 4), so we use training loss primarily to compare adaptation speed in the

¹<https://huggingface.co/collections/nvidia/nemotron-pre-training-datasets>

Table 1: **Evaluated models.** Backbones are hybrid (H) or full (F) attention.

model	attn	checkpoint	parameters			
			trunk	embed	head	total
<i>ours</i>						
dQwen3.5-0.8B	H	UT-IFML/dQwen3.5-0.8B-Base	0.50B	0.25B	tied	0.75B
dQwen3.5-2B	H	UT-IFML/dQwen3.5-2B-Base	1.37B	0.51B	tied	1.88B
dQwen3.5-4B	H	UT-IFML/dQwen3.5-4B-Base	3.57B	0.64B	tied	4.21B
dQwen3.5-9B	H	UT-IFML/dQwen3.5-9B-Base	6.92B	1.02B	1.02B	8.95B
dQwen3-1.7B	F	UT-IFML/dQwen3-1.7B-Base	1.41B	0.31B	tied	1.72B
<i>diffusion comparators</i>						
CoDA	F	Salesforce/CoDA-v0-Base	1.41B	0.31B	0.31B	2.03B
LLaDA-8B	F	GSAI-ML/LLaDA-8B-Base	6.98B	0.52B	0.52B	8.02B
Dream-7B	F	Dream-org/Dream-v0-Base-7B	6.53B	0.54B	0.54B	7.62B
Dream-Coder-7B	F	Dream-org/Dream-Coder-v0-Base-7B	6.53B	0.54B	0.54B	7.62B
<i>autoregressive parents</i>						
Qwen3.5-0.8B	H	Qwen/Qwen3.5-0.8B	0.50B	0.25B	tied	0.75B
Qwen3.5-2B	H	Qwen/Qwen3.5-2B	1.37B	0.51B	tied	1.88B
Qwen3.5-4B	H	Qwen/Qwen3.5-4B	3.57B	0.64B	tied	4.21B
Qwen3.5-9B	H	Qwen/Qwen3.5-9B	6.92B	1.02B	1.02B	8.95B
Qwen3-1.7B	F	Qwen/Qwen3-1.7B	1.41B	0.31B	tied	1.72B
Qwen2.5-7B	F	Qwen/Qwen2.5-7B	6.53B	0.54B	0.54B	7.62B
Qwen2.5-Coder-7B	F	Qwen/Qwen2.5-Coder-7B	6.53B	0.54B	0.54B	7.62B

closely matched hybrid/control setting rather than as a universal measure of capability. All runs begin from the same mixture under the same seed, although wall-clock limits require periodic resumes that reshuffle the data and cause their sample streams to diverge. We therefore evaluate fixed 50B- and 100B-token checkpoints rather than train to convergence. Loss is still declining at the end of every run, but prior work has found that longer AR-to-DLM adaptation can nevertheless degrade downstream performance [Gong et al., 2026].

4 Evaluation

We evaluate whether a mostly causal hybrid backbone can nevertheless become an effective DLM. We focus on two questions: whether hybrid backbones provide efficient starting points for AR-to-DLM adaptation, and whether the resulting models retain the parallel and any-order decoding behavior that defines DLMs. We evaluate downstream capability on standard knowledge, mathematics, and coding benchmarks: MMLU [Hendrycks et al., 2021a], GSM8K [Cobbe et al., 2021], MATH500 [Hendrycks et al., 2021b, Lightman et al., 2024], HumanEval [Chen et al., 2021], and MBPP [Austin et al., 2021b], along with the EvalPlus variants of the last two [Liu et al., 2023]; complete evaluation details are in Section C.

Models. Table 1 lists all evaluated models and their parameter decompositions. We compare base DLM checkpoints throughout, including our own models before post-training. Post-training can substantially change downstream performance and is highly recipe-sensitive, so we treat it as a separate problem from AR-to-DLM adaptation. We use trunk (non-embedding, non-head) parameters as a proxy for capacity, yielding comparison groups at the ≈ 1.4 B and 6.5–7B scales.

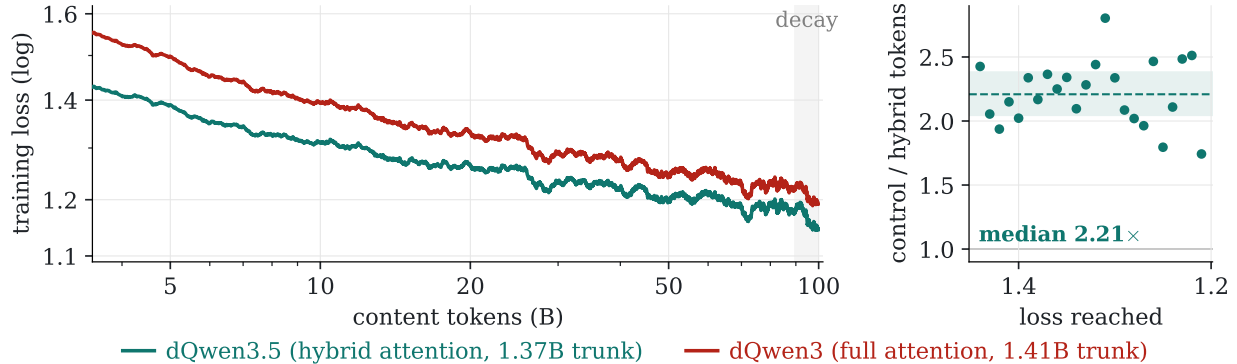


Figure 5: **Hybrid attention reaches the same loss in about half the tokens.** dQwen3.5-2B (1.37B trunk) against dQwen3-1.7B (1.41B trunk). *Left*: training loss after 2B warmup tokens. *Right*: at each stable-phase loss level, the ratio of tokens each run needed to first reach it.

Dream-7B, Dream-Coder-7B, and CoDA are themselves adapted from Qwen2.5-7B [Qwen Team et al., 2024], Qwen2.5-Coder-7B [Hui et al., 2024], and Qwen3-1.7B [Yang et al., 2025a], respectively; the last is also the AR model used for our control. CoDA unties its embedding and output head, giving 2.03B total parameters despite a 1.41B trunk.

Decoding. We evaluate all DLMs under a common decoding scheme rather than their released inference recipes. By default, we use a generation canvas of size 1024 and a block size of 32 for block decoding. For parallel decoding, we use either a fixed decoding rate or confidence thresholding. For confidence thresholding, we unmask all positions above the threshold, while always unmasking at least enough highest-confidence positions to satisfy the linear schedule $\alpha_t = 1 - t$. Otherwise, we greedily unmask only the single most confident position at each step.

4.1 Hybrid backbones adapt faster

We first ask whether a hybrid backbone provides a more efficient starting point for AR-to-DLM adaptation. We compare dQwen3.5-2B against dQwen3-1.7B under the same adaptation recipe, with closely matched trunks of 1.37B and 1.41B parameters, respectively. Importantly, the architectural intervention is very different: only 6/24 sequence-processing layers are bidirectionalized in dQwen3.5-2B, compared with all 28 layers in the full-attention control. This pair therefore gives our closest test of whether retaining a causal recurrent majority can make adaptation more efficient.

Against this control, the hybrid reaches the diffusion objective substantially faster (Figure 5). Across stable-phase loss levels reached by both runs, the control requires a median $2.21\times$ as many content tokens to reach the same loss. We interpret this as faster adaptation to the diffusion objective, rather than as evidence that training loss directly measures downstream capability. The comparison is not exact because the tokenizers differ: on our mixture, dQwen3 averages 4.18 bytes/token versus 4.05 for dQwen3.5, which complicates a direct comparison of per-token losses.

The downstream results provide a second positive signal. Before adaptation, Qwen3.5-2B underperforms Qwen3-1.7B on all 7 benchmarks we evaluate, yet after 50B adaptation tokens dQwen3.5-2B overtakes the full-attention control on 3/7, including HumanEval, MBPP, and MBPP+ (Table 2).

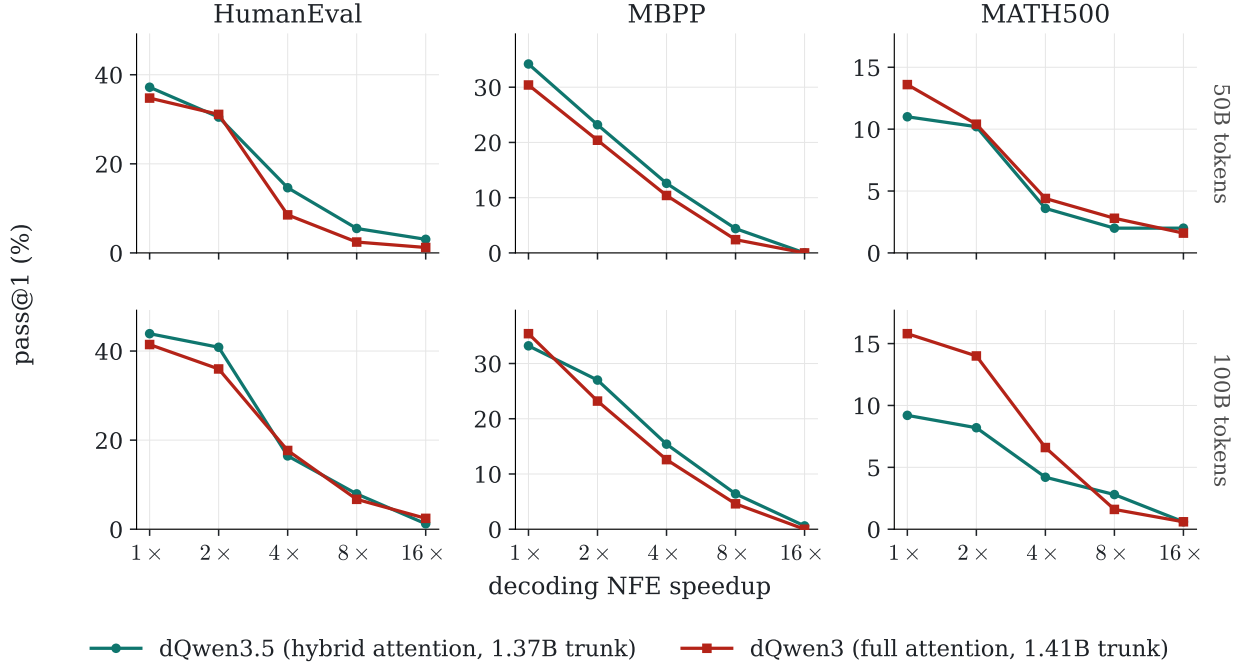


Figure 6: **Hybrid adapts faster and can parallelize.** Accuracy against decoding speedup, where $1\times$ is one token per step and 1024 NFEs. Notably, Qwen3.5-2B (AR, hybrid) actually underperforms Qwen3-1.7B (AR, full) on every benchmark (Table 2).

The hybrid also often remains ahead on code under parallel decoding (Figure 6). It remains weaker on mathematics, and these downstream differences cannot be attributed solely to the backbone because the AR parents already differ substantially and our code-heavy adaptation mixture also affects performance (Section B). Nevertheless, reversing several initially unfavorable comparisons provides further evidence that the hybrid is an effective starting point for adaptation.

4.2 50B tokens is enough for adaptation

A central goal of AR-to-DLM adaptation is to obtain useful diffusion capability with substantially less training than pretraining from scratch. We therefore evaluate every dQwen3.5 size after both 50B and 100B adaptation tokens (Table 2). Across scales, the 50B checkpoints are already strong, and extending training to 100B tokens does not uniformly improve downstream performance, even as training loss continues to fall.

Interestingly, the benefit of longer adaptation decreases with model size. At 0.8B, the 100B checkpoint improves 5/7 benchmarks over its 50B counterpart. At 9B, however, it improves only 1/7, while the 50B checkpoint performs better on the remaining 6/7. The regressions are concentrated in knowledge and mathematics, whereas code more often benefits from continued adaptation, consistent with our code-heavy training mixture and the mixture sensitivity observed in Section B. Thus, particularly at larger scales, useful AR-to-DLM adaptation emerges by 50B tokens, and further training under the same recipe can be counterproductive.

Table 2: **Too much adaptation can hurt.** Each model at its 50B- and 100B-token checkpoints, per Section 3. We bold the higher of each 50B/100B pair.

model	tokens	MMLU	GSM8K	MATH500	HEval	HEval+	MBPP	MBPP+
Qwen3.5-0.8B	—	50.31	34.95	17.00	22.56	20.73	17.40	24.07
dQwen3.5-0.8B	50B	32.62	8.49	5.40	28.05	25.61	22.60	29.10
	100B	30.24	8.11	6.20	35.37	31.71	23.20	32.28
Qwen3.5-2B	—	57.23	58.98	30.80	37.20	34.15	23.40	31.75
dQwen3.5-2B	50B	45.50	20.55	11.00	37.20	32.32	34.20	40.48
	100B	41.55	19.79	9.20	43.90	38.41	33.20	43.12
Qwen3.5-4B	—	69.82	85.29	49.60	59.15	53.66	40.20	48.41
dQwen3.5-4B	50B	62.42	57.77	25.00	54.88	51.83	44.60	51.32
	100B	57.95	53.45	25.60	60.37	57.32	44.60	50.00
Qwen3.5-9B	—	69.85	87.34	48.80	68.90	62.80	55.00	63.76
dQwen3.5-9B	50B	74.57	74.98	37.40	64.02	59.15	50.20	58.20
	100B	72.78	69.98	36.80	62.20	58.54	52.00	56.61
Qwen3-1.7B	—	60.30	74.75	48.40	40.24	37.80	43.80	46.03
dQwen3-1.7B	50B	51.30	42.23	13.60	34.76	33.54	30.40	38.89
	100B	48.12	39.35	15.80	41.46	37.20	35.40	46.83

4.3 Hybrid DLMs can decode in any order, even though RNNs are causal

A central concern with retaining causal RNN layers is that the resulting DLM might simply inherit a left-to-right decode order. We therefore study decode trajectories on HumanEval, recording the decoding step d_k on which each canvas position k is decoded. We use a canvas of size 256 because HumanEval solutions are short, and a longer canvas would mostly measure padding (Section C.6). To distinguish local from global ordering, we define

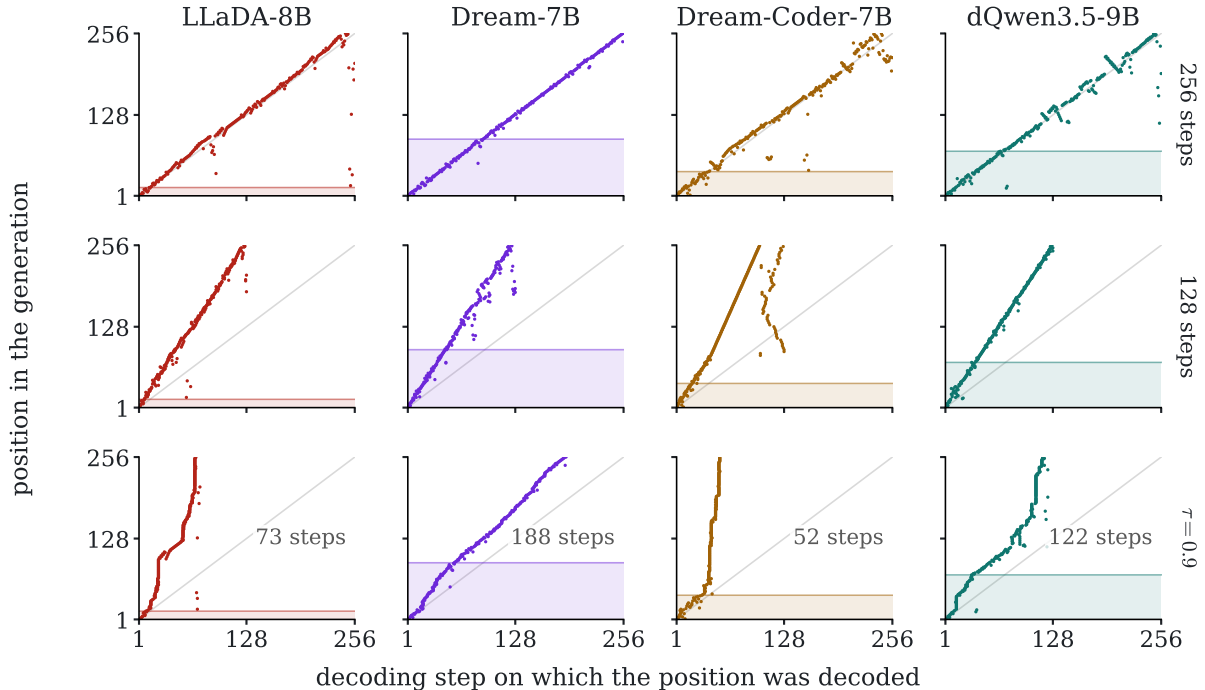
$$\text{ARL} = \frac{|\{k : d_k < d_{k+1}\}|}{|\{k : d_k \neq d_{k+1}\}|}, \quad \text{ARG} = \frac{|\{i < j : d_i < d_j\}|}{|\{i < j : d_i \neq d_j\}|}.$$

Here, local AR-ness (ARL) measures left-to-right ordering among adjacent positions, while global AR-ness (ARG) measures it across all position pairs. Random order scores 0.5 on both metrics, whereas strict AR decoding scores 1.0.

As shown in Figure 7, all four DLMs are globally left-to-right but locally much more flexible. Under full-canvas decoding, dQwen3.5-9B has local AR-ness 0.636, squarely within the 0.631–0.652 range of the three full-attention DLMs, despite retaining causal RNNs in most of its layers. At the same time, all models have high global AR-ness, ranging from 0.884 to 0.967. This local–global distinction is consistent with prior observations that even full-attention DLMs, including models trained from scratch, tend to globally decode from left to right [Gong et al., 2026]. Thus, the causal RNN layers in dQwen3.5 do not prevent the local out-of-order decoding behavior characteristic of full-attention DLMs. We report additional decoding schemes, models, and trajectories in Section C.6.

4.4 Hybrid DLMs can decode in parallel

Having established that causal RNN layers do not prevent local any-order decoding, we next ask whether this flexibility translates into parallel generation. Figure 8 evaluates dQwen3.5-9B



model	full-canvas 256		full-canvas 128		full-canvas $\tau = 0.9$		
	ARL	ARG	ARL	ARG	ARL	ARG	NFEs
LLaDA-8B	0.631	0.932	0.617	0.924	0.630	0.945	83 ± 38
Dream-7B	0.638	0.884	0.619	0.830	0.659	0.891	139 ± 40
Dream-Coder-7B	0.652	0.937	0.607	0.934	0.662	0.944	136 ± 55
dQwen3.5-9B	0.636	0.967	0.651	0.970	0.655	0.975	129 ± 34
AR decoder	1.000	1.000	1.000	1.000	1.000	1.000	256

Figure 7: **Decode orders for the 6.5–7B trunks on HumanEval1/58.** We use canvas size 256 because extracted solutions (shaded) tend to be short. *Top*: decode position vs. step number; each dot marks when and where a position is unmasked. *Bottom*: ARL and ARG over all 164 HumanEval problems. All DLMs decode globally left-to-right but remain locally flexible, including the mostly causal dQwen3.5-9B and the from-scratch LLaDA-8B. See Section C.6 for additional results.

against the other 6.5–7B-trunk DLMs as decoding is increasingly parallelized. We consider both fixed budgets, which compare models at matched NFE counts, and confidence thresholding, which allows each model to choose how aggressively to decode in parallel. Despite retaining a causal recurrent majority, dQwen3.5-9B maintains a strong accuracy–speed trade-off under both schemes.

Under fixed budgets, dQwen3.5-9B leads all comparators on HumanEval at every speedup beyond $1\times$, while remaining competitive on MBPP and MATH500. Confidence thresholding yields the same broader picture, with dQwen3.5-9B remaining on a competitive quality–speed frontier across several-fold NFE speedups. Thus, retaining causal recurrence in most of the backbone does not prevent the adapted model from realizing the parallel decoding behavior expected of a DLM. Additional models and decoding settings are reported in Section C.7.

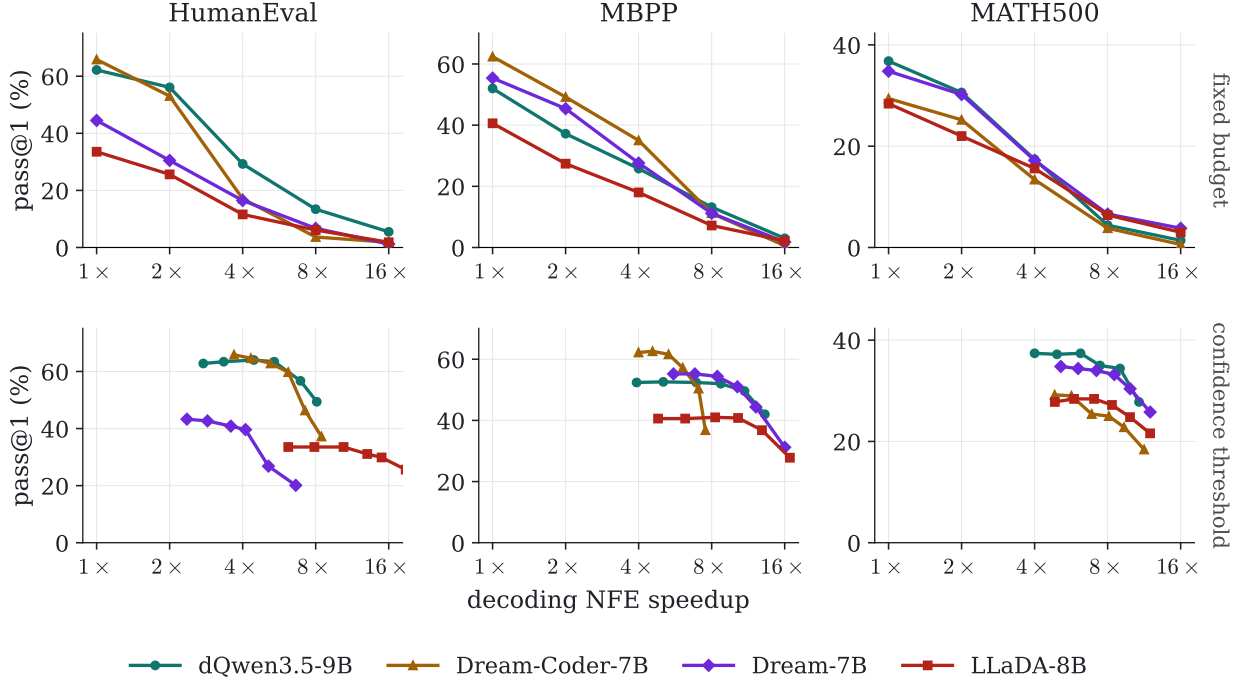


Figure 8: **Parallel decoding for the 6.5–7B trunks.** We use full-canvas decoding with size 1024; $1\times$ denotes one token per step, or 1024 NFEs, and larger speedups use proportionally fewer NFEs. *Top:* fixed decoding budgets, showing how performance degrades at faster speeds (fewer steps). *Bottom:* adaptive confidence thresholding with $\tau \in \{0.5, 0.6, 0.7, 0.8, 0.9, 0.95\}$.

4.5 dQwen3.5 is a competitive model family

Having established that hybrid backbones adapt efficiently while attaining the decoding behavior expected of DLMs, we next ask how they compare on standard benchmarks. Table 3 evaluates base DLMs under the same full-canvas, one-token-per-step decoding scheme, grouped by trunk size.

Despite using substantially fewer training tokens, dQwen3.5 is competitive across both comparison scales. At the $\approx 1.4\text{B}$ trunk scale, dQwen3.5-2B after 50B adaptation tokens outperforms CoDA, trained for 200B tokens, on 6/7 benchmarks. At the 6.5–7B scale, dQwen3.5-9B after 50B tokens attains the best DLM score on 4/7 benchmarks against Dream-7B (580B tokens), Dream-Coder-7B (322B), and LLaDA-8B (2.3T). Together with the preceding results, this suggests that retaining a mostly causal hybrid backbone need not trade away either DLM-like decoding behavior or down-stream capability. We report the remaining model sizes and block-decoding results in Section C.5.

5 Discussion

Why might hybrid attention suffice for diffusion? Although each GDN is causal, bidirectional attention injects future context into representations processed by downstream layers. This sparse bidirectional mixing may suffice if non-causal dependence in language is itself comparatively selective. Suppose, for example, that language modeling consists mostly of causal dependencies, except for a small set of “anchor” tokens [Rout et al., 2025] that carry relevant future context, such

Table 3: **dQwen3.5 is competitive with DLMs trained on far more tokens.** Size classes are set by trunk parameters. We bold the best DLM number in each column of a class. Other sizes in Table 10.

model	trunk	tokens	MMLU	GSM8K	MATH500	HEval	HEval+	MBPP	MBPP+
<i>≈1.4B trunk</i>									
Qwen3-1.7B	1.41B	—	60.30	74.75	48.40	40.24	37.80	43.80	46.03
Qwen3.5-2B	1.37B	—	57.23	58.98	30.80	37.20	34.15	23.40	31.75
CoDA	1.41B	200B	26.21	2.65	3.80	25.00	20.12	33.80	41.01
dQwen3-1.7B	1.41B	50B	51.30	42.23	13.60	34.76	33.54	30.40	38.89
dQwen3-1.7B	1.41B	100B	48.12	39.35	15.80	41.46	37.20	35.40	46.83
dQwen3.5-2B	1.37B	50B	45.50	20.55	11.00	37.20	32.32	34.20	40.48
dQwen3.5-2B	1.37B	100B	41.55	19.79	9.20	43.90	38.41	33.20	43.12
<i>6.5–7B trunk</i>									
Qwen2.5-7B	6.53B	—	74.13	79.61	40.80	55.49	48.78	64.40	69.84
Qwen2.5-Coder-7B	6.53B	—	68.07	78.77	37.20	59.76	53.66	68.40	71.43
Qwen3.5-9B	6.92B	—	69.85	87.34	48.80	68.90	62.80	55.00	63.76
LLaDA-8B	6.98B	2.3T	65.88	71.34	28.40	33.54	28.66	40.60	45.50
Dream-7B	6.53B	580B	71.45	74.91	34.80	44.51	38.41	55.40	57.67
Dream-Coder-7B	6.53B	322B	65.41	72.18	29.40	65.85	57.93	62.40	65.08
dQwen3.5-9B	6.92B	50B	74.57	74.98	37.40	64.02	59.15	50.20	58.20
dQwen3.5-9B	6.92B	100B	72.78	69.98	36.80	62.20	58.54	52.00	56.61

as a later noun or verb that disambiguates an earlier phrase. Let $A \subset [L]$ with $|A| \ll L$ denote such positions, and consider the factorization $q(x_1, \dots, x_L) = q(x_A) \prod_{i \notin A} q(x_i \mid x_{<i}, x_A)$. Under this view, the x_A tokens capture the non-causal dependence, while every other token is autoregressive in its prefix once x_A is known. The causal RNN layers can therefore model the prefix dependence, while the bidirectional attention layers need only provide access to a few future positions. When $|A| \ll L$, this gives an interpretation for why a small number of bidirectional attention layers may suffice within a mostly causal recurrent backbone.

AR-to-DLM adaptation. AR-to-DLM adaptation remains expensive, with many models trained for hundreds of billions of additional tokens. Our results suggest that the backbone itself can be one important factor: dQwen3.5 changes the information flow in only a minority of its layers, while the recurrent majority remains causal. In our matched-scale comparison, this hybrid reaches the same diffusion training loss in roughly half the tokens required by the full-attention control, suggesting that modern hybrid AR models may be particularly efficient starting points for adaptation.

Limitations. Our direct architectural comparison is limited to one pair at one scale, dQwen3.5-2B and dQwen3-1.7B, whose trunks are closely matched but whose parent models and tokenizers differ. Thus, the training-loss comparison is informative about adaptation speed, but downstream differences cannot be attributed solely to backbone architecture. Our training mixture may also be further tuned, and our development experiments show that mixture choice can substantially alter what capabilities survive adaptation (Section B). Finally, we evaluate base DLMs only, as post-training (SFT, RL) substantially affects benchmark performance and is itself recipe-sensitive.

Future work. Our results suggest that useful adaptation may be possible at substantially less than 50B tokens, especially at larger scales (Section 4.2). A future study should pretrain matched

hybrid and full-attention AR models from scratch, then adapt both under the same tokenizer, data, and optimization recipe. This would isolate whether the faster adaptation we observe is genuinely caused by hybrid structure and clarify how the fraction and placement of RNN layers affect the trade-off. More broadly, understanding which parts of an AR backbone are critical for diffusion language modeling will enable more efficient adaptations.

Acknowledgments. This research has been supported by NSF Grants 2505865 and 2112471, the UT Austin Machine Learning Lab, and computing support on the Vista GPU Cluster through the Center for Generative AI (CGAI) and the Texas Advanced Computing Center (TACC) at UT Austin.

References

- Marianne Arriola, Aaron Gokaslan, Justin T Chiu, Zhihan Yang, Zhixuan Qi, Jiaqi Han, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021a.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021b.
- Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. LLaDA 2.0: Scaling up diffusion language models to 100B. *arXiv preprint arXiv:2512.15745*, 2025.
- Haolin Chen, Shiyu Wang, Can Qin, Bo Pang, Zuxin Liu, Jieli Qiu, Jianguo Zhang, Yingbo Zhou, Zeyuan Chen, Ran Xu, Shelby Heinecke, Silvio Savarese, Caiming Xiong, Huan Wang, and Weiran Yao. CoDA: Coding LM via diffusion adaptation. *arXiv preprint arXiv:2510.03270*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob

- McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Shuang Cheng, Yihan Bian, Dawei Liu, Yuhua Jiang, Yihao Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. SDAR: A synergistic Diffusion-AutoRegression paradigm for scalable sequence generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, Chenhan Yu, Ye Yu, Weijia Chen, Sajad Norouzi, Jingyu Liu, Shiyi Lan, Ligeng Zhu, Jin Wang, Jindong Jiang, Morteza Mardani, Mehran Maghoumi, Song Han, Ante Jukić, Nima Tajbakhsh, Jan Kautz, and Pavlo Molchanov. Nemotron-Labs-Diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. *arXiv preprint arXiv:2607.05722*, 2026a.
- Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. Efficient-DLM: From autoregressive to diffusion language models, and beyond in speed. In *International Conference on Machine Learning (ICML)*, 2026b.
- Shansan Gong, Shivam Agarwal, Yizhe Zhang, Jiacheng Ye, Lin Zheng, Mukai Li, Chenxin An, Peilin Zhao, Wei Bi, Jiawei Han, Hao Peng, and Lingpeng Kong. Scaling diffusion language models via adaptation from autoregressive models. In *International Conference on Learning Representations (ICLR)*, 2025.
- Shansan Gong, Ruixiang Zhang, Huangjie Zheng, Jiatao Gu, Navdeep Jaitly, Lingpeng Kong, and Yizhe Zhang. DiffuCoder: Understanding and improving masked diffusion models for code generation. In *International Conference on Learning Representations (ICLR)*, 2026.
- Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L. Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014*, 2023.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021b.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

- Inception Labs, Samar Khanna, Siddhant Kharbanda, Shufan Li, Harshit Varma, Eric Wang, Sawyer Birnbaum, Ziyang Luo, Yanis Miraoui, Akash Palrecha, Stefano Ermon, Aditya Grover, and Volodymyr Kuleshov. Mercury: Ultra-fast language models based on diffusion. *arXiv preprint arXiv:2506.17298*, 2025.
- Rabeeh Karimi Mahabadi, Sanjeev Satheesh, Shrimai Prabhumoye, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-CC-Math: A 133 billion-token-scale high quality math pretraining dataset. In *International Conference on Learning Representations (ICLR)*, 2026.
- Urvashi Khandelwal, He He, Peng Qi, and Dan Jurafsky. Sharp nearby, fuzzy far away: How neural language models use context. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- Xiang Lisa Li, John Thickstun, Ishaan Gulrajani, Percy Liang, and Tatsunori B. Hashimoto. Diffusion-LM improves controllable text generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid Transformer-Mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. In *International Conference on Machine Learning (ICML)*, 2024.
- MiniMax. MiniMax-01: Scaling foundation models with lightning attention. *arXiv preprint arXiv:2501.08313*, 2025.
- Shen Nie, Fengqi Zhu, Chao Du, Tianyu Pang, Qian Liu, Guangtao Zeng, Min Lin, and Chongxuan Li. Scaling up masked diffusion models on text. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025b.
- Shen Nie, Qiyang Min, Shaoxuan Xu, Zihao Huang, Yuxuan Song, Yong Shan, Yankai Lin, Wayne Xin Zhao, Chongxuan Li, and Ji-Rong Wen. Improved large language diffusion models. *arXiv preprint arXiv:2606.25331*, 2026.
- NVIDIA. Nemotron 3 Nano: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. *arXiv preprint arXiv:2512.20848*, 2025a.

- NVIDIA. NVIDIA Nemotron Nano 2: An accurate and efficient hybrid Mamba-Transformer reasoning model. *arXiv preprint arXiv:2508.14444*, 2025b.
- Joe O’Connor and Jacob Andreas. What context features can Transformer language models use? In *Annual Meeting of the Association for Computational Linguistics and the International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021.
- Qwen Team. Qwen3.5: Towards native multimodal agents. <https://qwen.ai/blog?id=qwen3.5>, 2026. Model series release; repository: <https://github.com/QwenLM/Qwen3.5>.
- Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Litu Rout, Constantine Caramanis, and Sanjay Shakkottai. Anchored diffusion language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Jiaxin Shi and Michalis K. Titsias. Demystifying diffusion objectives: Reweighted losses are better variational bounds. *arXiv preprint arXiv:2511.19664*, 2025.
- Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-CC: Transforming Common Crawl into a refined long-horizon pretraining dataset. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.
- Simeng Sun, Kalpesh Krishna, Andrew Mattarella-Micke, and Mohit Iyyer. Do long-range language models actually use long-range context? In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- Zhihui Xie, Jiacheng Ye, Lin Zheng, Jiahui Gao, Jingwei Dong, Zirui Wu, Xueliang Zhao, Shansan Gong, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream-Coder 7B: An open diffusion language model for code. *arXiv preprint arXiv:2509.01142*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren,

- Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025b.
- Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.
- Yuchen Zhu, Jing Shi, Chongjian Ge, Hao Tan, Yiran Xu, Wanrong Zhu, Jason Kuen, Koustava Goswami, Rajiv Jain, Yongxin Chen, Molei Tao, and Jiuxiang Gu. FLARE: Diffusion for hybrid language model. *arXiv preprint arXiv:2606.01774*, 2026.

A Additional background on masked diffusion language models

Notation and forward process. Let V be the vocabulary, and let $x = (x^1, \dots, x^L)$ denote the clean tokens on an L -length generation canvas, with any prompt tokens held fixed and suppressed from the notation. We represent each token x^k and the mask token m as one-hot vectors in $\mathbb{R}^{|V|}$. Masked diffusion gradually replaces canvas tokens with m according to a monotonically decreasing survival schedule $\alpha_t \in [0, 1]$, with $\alpha_0 = 1$ and $\alpha_1 = 0$ [Austin et al., 2021a, Sahoo et al., 2024, Shi et al., 2024]. The forward marginal factorizes across positions,

$$q(z_t | x) = \prod_{k=1}^L q(z_t^k | x), \quad q(z_t^k | x) = \text{Cat}\left(z_t^k; \alpha_t x^k + (1 - \alpha_t)m\right).$$

Thus, at time t , each canvas position remains clean with probability α_t and is masked with probability $1 - \alpha_t$. We use the linear schedule $\alpha_t = 1 - t$, so sampling $t \sim U(0, 1)$ samples the masking ratio uniformly.

For a uniformly spaced T -step discretization of $[0, 1]$, let $t \in \{\frac{1}{T}, \frac{2}{T}, \dots, 1\}$ and $s = t - \frac{1}{T}$ denote the preceding time. The corresponding forward transition is

$$q(z_t^k | z_s^k) = \text{Cat}\left(z_t^k; \frac{\alpha_t}{\alpha_s} z_s^k + \left(1 - \frac{\alpha_t}{\alpha_s}\right)m\right), \quad z_0^k = x^k.$$

Once a position becomes masked it remains masked, so m is an absorbing state.

Reverse process. If the clean sequence x were known, the exact reverse transition is

$$q(z_s^k | z_t, x) = \begin{cases} \text{Cat}(z_s^k; z_t^k), & z_t^k \neq m, \\ \text{Cat}\left(z_s^k; \frac{\alpha_s - \alpha_t}{1 - \alpha_t} x^k + \frac{1 - \alpha_s}{1 - \alpha_t} m\right), & z_t^k = m. \end{cases}$$

Thus, an unmasked position is copied unchanged, while a masked position is decoded to its clean value with probability $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

At generation time, x is unknown, so the model predicts a clean-token distribution $p_\theta^k(\cdot | z_t) \in \mathbb{R}^{|V|}$ at each position k . Substituting this prediction for x^k gives

$$p_\theta(z_s | z_t) := \prod_{k=1}^L p_\theta(z_s^k | z_t), \quad p_\theta(z_s^k | z_t) := \begin{cases} \text{Cat}(z_s^k; z_t^k), & z_t^k \neq m, \\ \text{Cat}\left(z_s^k; \frac{\alpha_s - \alpha_t}{1 - \alpha_t} p_\theta^k(\cdot | z_t) + \frac{1 - \alpha_s}{1 - \alpha_t} m\right), & z_t^k = m. \end{cases}$$

Generation begins with the prompt fixed and the L -length canvas fully masked, then repeatedly applies these reverse transitions until the canvas is decoded.

Training objective. For the absorbing-mask process above, the parameter-dependent part of the standard negative evidence lower bound (NELBO) reduces to

$$\mathcal{L}_{\text{NELBO}}(x; \theta) = \sum_t \mathbb{E}_{z_t \sim q(\cdot | x)} \lambda_t \sum_{k: z_t^k = m} \left[-\log p_\theta^k(x^k | z_t) \right], \quad \lambda_t = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}, \quad (1)$$

up to terms independent of θ [Austin et al., 2021a, Sahoo et al., 2024, Shi et al., 2024]. Here, λ_t is the probability that a masked position is decoded in the exact reverse transition from t to s .

More generally, each time may be reweighted by a factor w_t ,

$$\mathcal{L}_w(x; \theta) = \sum_t \mathbb{E}_{z_t \sim q(\cdot | x)} w_t \lambda_t \sum_{k: z_t^k = m} \left[-\log p_\theta^k(x^k | z_t) \right].$$

Following the *simple weighting* of Shi and Titsias [2025], we take $w_t = 1/\lambda_t$, so the schedule-dependent factor cancels and every masked target receives equal weight. Under our linear schedule, we sample $t \sim U(0, 1)$ directly and train with

$$\mathcal{L}(x; \theta) = \mathbb{E}_{t \sim U(0, 1)} \mathbb{E}_{z_t \sim q(\cdot | x)} \frac{1}{L} \sum_{k: z_t^k = m} \left[-\log p_\theta^k(x^k | z_t) \right].$$

This is the training objective used throughout our adaptation experiments. Because it differs from the standard NELBO, its value is not directly a likelihood bound [Shi et al., 2024, App. H.4].

Remark. The standard NELBO places more per-target weight when the sequence is lightly masked. Canceling λ_t instead weights every masked target equally, shifting more total training weight toward highly masked states encountered early in generation. Related reweightings retain a variational interpretation [Shi and Titsias, 2025].

B Training details

B.1 Training configuration

Table 4 gives the full training mixture with provenance, with subsets written as parent / subset. Despite its name, Nemotron-Pretraining-SFT-v1 contains SFT-style data released for blending into pretraining, and we use it accordingly. Table 5 gives the complete optimization configuration.

Table 4: **Training data mixture.** Token shares of the 14 subsets, grouped by source repository within NVIDIA’s Nemotron pretraining releases [Karimi Mahabadi et al., 2026, NVIDIA, 2025a,b, Su et al., 2025]; only the nvidia/ organization prefix is elided. The two verified rows are execution-verified derivations of Code-Concepts.

family	source	token share (%)
code (50%)	Nemotron-Pretraining-Code-v2 / Synthetic-Code:	
	synthetic-student-teacher	7
	synthetic-rewriting	6
	synthetic-question-answering	4
	synthetic-code-review	3
	Nemotron-Pretraining-Specialized-v1.1:	
	Code-Concepts (verified, completion)	5
	Code-Concepts (verified, instruct)	5
math (15%)	Nemotron-Pretraining-SFT-v1 / SFT-Code	12
	Nemotron-CC-Code-v1	8
	Nemotron-Pretraining-Specialized-v1 / Math-Textbooks	10
general (35%)	Nemotron-CC-Math-v1 / 4plus_MIND	5
	Nemotron-Pretraining-Specialized-v1:	
	Wiki-Rewrite	5
	InfiniByte-Reasoning	5
	Nemotron-CC-v2.1:	
	High-Quality	20
	High-Quality-DQA	5

Code preprocessing. For the Code-Concepts subset, we additionally verify the released solutions against examples provided in their docstrings. We execute at most three declared examples per row and retain only solutions that pass, leaving 53.4% of the original pool. We render each surviving solution in two forms. The *completion form* maps a function signature to its body, matching tasks such as HumanEval. Because MBPP instead provides a natural-language specification, we also construct an *instruction form*: the docstring prose is spliced into a short instruction, with its verified examples appearing in a flush-left >>> preamble.

Augmentations. Sequences are packed to length 4096 and lightly augmented. With probability 0.05, an entire batch is truncated to a shorter length; with probability 0.10, a sample retains a clean prefix; and with probability 0.05, we append a supervised padding suffix. A quarter of the clean-prefix cases are instead trained fully masked, corresponding to $\sim 2.5\%$ of rows overall. Truncated lengths are sampled on a 256-token grid with mean 2048. Together, these augmentations

Table 5: **Optimization configuration, as run.** Identical across sizes except the learning rate. The 25k-step branch is the same schedule with decay starting at step 20,000. Unlisted settings are HuggingFace Trainer defaults.

optimizer	AdamW ($\beta_1=0.9, \beta_2=0.95$)			
weight decay	0.01			
gradient clip	1.0			
learning rate	0.8B: 3×10^{-5}	2B: 2×10^{-5}	4B: 2×10^{-5}	9B: 1×10^{-5}
schedule	linear warmup (1k steps), stable (44k), cosine decay to 0 (5k)			
batch	512 sequences $\times$ 4096 tokens (~ 2 M content tokens/step)			
precision	bf16 (tf32 matmul)			

leave about 3.9k content tokens per row on average, or ~ 2 M content tokens in a 512-row step. Thus, 25k and 50k training steps correspond to approximately 50B and 100B content tokens, respectively.

B.2 Telemetry

Every released run logs training loss, masked-token accuracy, and pre-clip gradient norm; Figure 9 shows all three over the full schedule. The traces are smooth across runs, with no obvious optimization instability. We include the gradient norm so that training stability can be inspected directly alongside the released telemetry. Loss is still falling at the end of every run, since we train to fixed 50B- and 100B-token budgets rather than to convergence. As discussed in Section 3.3, longer adaptation can nevertheless degrade downstream performance.

B.3 Development notes: learning rate and mixture

Learning-rate calibration. Adaptation is sensitive to learning rate in ways that training loss alone does not reveal. In preliminary single-seed probes, we swept several learning rates and measured how much of each source model’s above-chance MMLU performance survived adaptation; these probes use a different decoding setup and are not directly comparable with the main tables. Notably, our initial 10^{-4} configuration caused the smaller models to lose nearly all of their above-chance MMLU performance, while the 9B model trained at a lower learning rate retained substantially more. We interpret this pattern as catastrophic forgetting from overly aggressive updates, consistent with prior adaptation and continual-pretraining studies [Fu et al., 2026b, Gupta et al., 2023, Ye et al., 2025]. We therefore selected learning rates by capability retention rather than training loss.

Mixture exploration. Our initial mixture contained 71% code and no general web text; we arrived at the released mixture through a series of small-scale probes. At matched budgets, the released mixture improves 19/20 benchmark cells over this initial version, with code performance improving even as the code share falls to 50%. Thus, the larger initial code share was not translating into stronger code capability. Mixture tuning was most impactful for smaller models.

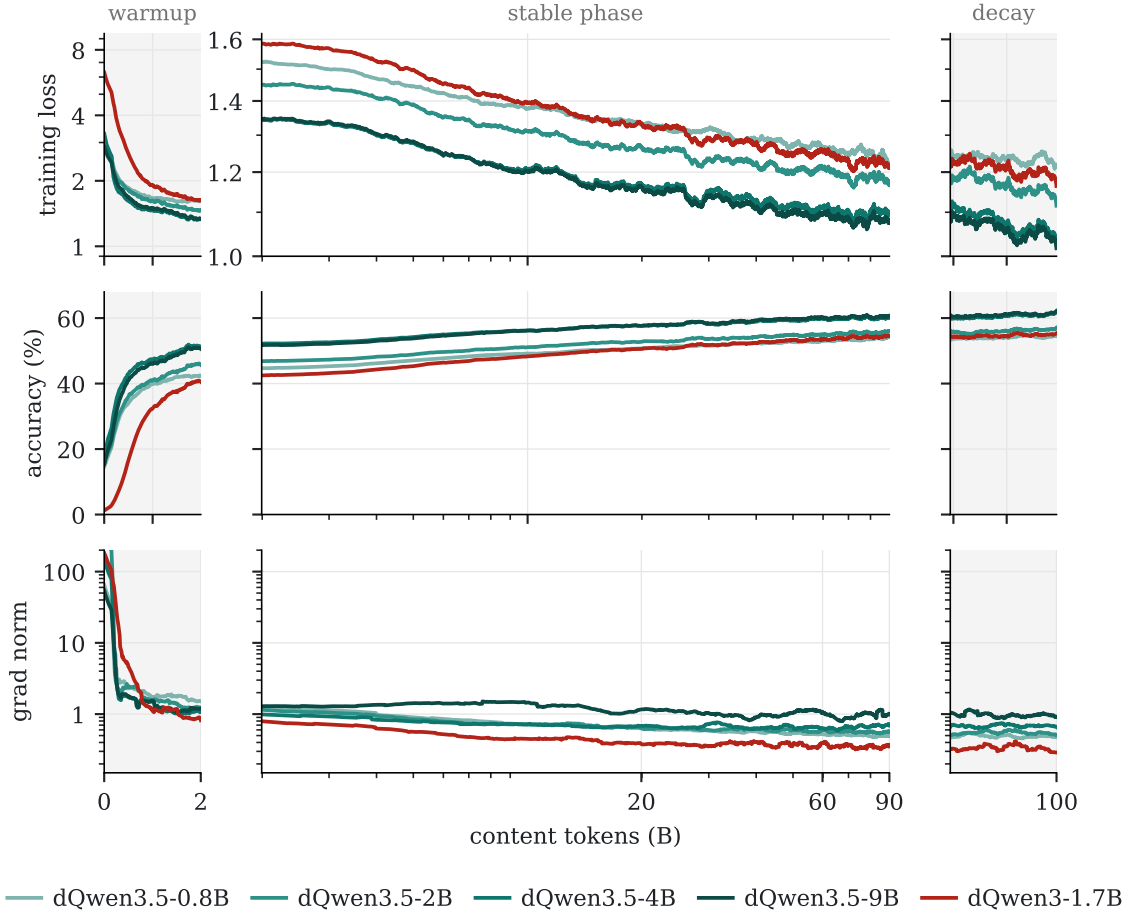


Figure 9: **Training telemetry for the released runs.** Loss, masked-token accuracy and pre-clip gradient norm for the four adapted sizes and the control, on aligned phase panels. Accuracy uses our own masking distribution (Section B.1), so its level is not comparable to other reports.

C Extended evaluation

C.1 Evaluation details

Table 6 lists the per-benchmark configuration. MMLU is multiple choice and is scored on the option’s single answer token, while MATH500 uses the `lm-eval` harness’s symbolic-equivalence check rather than string matching. Unless otherwise specified, DLM generation uses a canvas of size 1024 with greedy highest-confidence-first unmasking and no remasking. We use the same decoding scheme across models rather than each model’s released inference recipe.

Table 6: **Per-benchmark configuration.** Generation length is the canvas size for every generative task. Problem counts are as graded. The plus (+) variants feature harder unit tests; only HumanEval+ keeps its base benchmark’s prompts.

benchmark	task	shots	problems	scoring
MMLU	mmlu	5	14,042	answer-token likelihood
GSM8K	gsm8k_cot	8	1,319	exact match
MATH500	minerva_math500	4	500	symbolic equivalence
HumanEval	humaneval	0	164	pass@1
HumanEval+	humaneval_plus_sound	0	164	pass@1
MBPP	mbpp	3	500	pass@1
MBPP+	mbpp_plus_full	3	378	pass@1
MBPP (fence)	mbpp_ticks	3	500	pass@1
MBPP+ (fence)	mbpp_plus_ticks	3	378	pass@1

C.2 Checking our evaluation harness

Table 7 compares our scores with those reported by each comparator’s authors for the same base checkpoints. Across 18 referenced evaluations, the median absolute difference is about two points, supporting the use of our re-implemented evaluation harness. Two code comparisons differ by roughly 13 points, Dream-7B on HumanEval and Dream-Coder-7B on MBPP, where their released evaluation scripts use different generation or post-processing choices. We nevertheless use one fixed decoding scheme throughout the main comparison.

Table 7: **A check on our evaluation harness.** We rescore each comparator under our own protocol and compare against what its authors published, to confirm our implementation is sound. We fix one setting for everyone rather than each model’s released sampler, which explains the gap.

benchmark	LLaDA-8B ^a			Dream-7B ^b			Dream-Coder-7B ^c			CoDA ^d		
	pub.	ours	Δ	pub.	ours	Δ	pub.	ours	Δ	pub.	ours	Δ
MMLU	65.9	65.88	−0.0	69.5	71.45	+2.0	65.6	65.41	−0.2	—	—	—
GSM8K	70.3	71.34	+1.0	77.2	74.91	−2.3	71.1	72.18	+1.1	—	—	—
HumanEval	35.4	33.54	−1.9	57.9	44.51	−13.4	66.5	65.85	−0.7	29.3	25.00	−4.3
HumanEval+	—	—	—	—	—	—	60.4	57.93	−2.5	23.8	20.12	−3.7
MBPP	40.0	40.60	+0.6	56.2	55.40	−0.8	75.9	62.40	−13.5	35.2	33.80	−1.4
MBPP+	—	—	—	—	—	—	61.6	65.08	+3.5	46.0	41.01	−5.0

Ours: MMLU 5-shot, GSM8K 8, HumanEval 0, MBPP 3. An empty cell means the paper does not report it.

^a arXiv:2502.09992v3, Table 1. Generates on a 1024-position canvas at one token per step, the closest of the four to our protocol; GSM8K and MBPP at 4 shots.

^b arXiv:2508.15487v1, Table 1. Generates 512 tokens at a sampling temperature, with stop-string truncation disabled and an external sanitizer on HumanEval; MBPP at 4 shots.

^c arXiv:2509.01142v1, Table 1. Generates 512 tokens zero-shot, and scores MBPP on the 378-problem EvalPlus set rather than the 500-problem split we use.

^d arXiv:2510.03270v1, Table 4. Caps generation at 768 tokens, zero-shot.

C.3 Comparison with AR parents

Table 8 reports each adapted DLM as a fraction of its own AR parent, helping separate capability inherited from the parent model from capability retained through adaptation. For example, dQwen3.5-9B’s lower GSM8K performance is not inherited from a weaker parent, since Qwen3.5-9B is the strongest AR parent in that comparison. On MBPP, the picture reverses: Qwen3.5-9B begins behind the other AR parents, while dQwen3.5-9B retains more of its parent’s performance.

Table 8: **Each adapted DLM relative to its AR parent.** We divide each DLM score by the corresponding AR parent’s score; for our models, we use the 100B checkpoints. Values above 1 indicate higher benchmark performance after adaptation.

model	AR parent	MMLU	GSM8K	MATH500	HEval	HEval+	MBPP	MBPP+
<i>≈0.5B trunk</i>								
dQwen3.5-0.8B	Qwen3.5-0.8B	0.601	0.232	0.365	1.568	1.530	1.333	1.341
<i>≈1.4B trunk</i>								
CoDA	Qwen3-1.7B	0.435	0.035	0.079	0.621	0.532	0.772	0.891
dQwen3-1.7B	Qwen3-1.7B	0.798	0.526	0.326	1.030	0.984	0.808	1.017
dQwen3.5-2B	Qwen3.5-2B	0.726	0.336	0.299	1.180	1.125	1.419	1.358
<i>≈3.6B trunk</i>								
dQwen3.5-4B	Qwen3.5-4B	0.830	0.627	0.516	1.021	1.068	1.109	1.033
<i>6.5–7B trunk</i>								
Dream-7B	Qwen2.5-7B	0.964	0.941	0.853	0.802	0.787	0.860	0.826
Dream-Coder-7B	Qwen2.5-Coder-7B	0.961	0.916	0.790	1.102	1.080	0.912	0.911
dQwen3.5-9B	Qwen3.5-9B	1.042	0.801	0.754	0.903	0.932	0.945	0.888

C.4 Prompt-format sensitivity

MBPP’s [BEGIN] / [DONE] scaffold is specific to its evaluation harness, so we rescore every DLM using a plain ````python` fence while holding the remaining decoding scheme fixed (Table 9). Prompt format changes scores modestly overall, with larger effects for smaller models. The relative picture is broadly unchanged, and the main text reports the default scaffold throughout.

Table 9: **MBPP is sensitive to its prompt format, and smaller models more so.** Each model scored under the stock [BEGIN] / [DONE] scaffold and under a plain ````python` fence, everything else held fixed. The gain from the fence falls with model size and reverses for the largest models.

model	trunk	tokens	MBPP			MBPP+		
			stock	fence	Δ	stock	fence	Δ
$\approx 0.5B$ trunk								
dQwen3.5-0.8B	0.50B	50B	22.60	23.80	+1.20	29.10	33.33	+4.23
dQwen3.5-0.8B	0.50B	100B	23.20	29.40	+6.20	32.28	37.04	+4.76
$\approx 1.4B$ trunk								
CoDA	1.41B	200B	33.80	37.00	+3.20	41.01	43.65	+2.64
dQwen3-1.7B	1.41B	50B	30.40	35.40	+5.00	38.89	42.33	+3.44
dQwen3-1.7B	1.41B	100B	35.40	36.60	+1.20	46.83	49.47	+2.64
dQwen3.5-2B	1.37B	50B	34.20	34.40	+0.20	40.48	42.59	+2.11
dQwen3.5-2B	1.37B	100B	33.20	36.00	+2.80	43.12	44.71	+1.59
$\approx 3.6B$ trunk								
dQwen3.5-4B	3.57B	50B	44.60	46.80	+2.20	51.32	53.70	+2.38
dQwen3.5-4B	3.57B	100B	44.60	46.80	+2.20	50.00	52.38	+2.38
6.5–7B trunk								
LLaDA-8B	6.98B	2.3T	40.60	39.20	-1.40	45.50	44.71	-0.79
Dream-7B	6.53B	580B	55.40	55.80	+0.40	57.67	59.52	+1.85
Dream-Coder-7B	6.53B	322B	62.40	63.80	+1.40	65.08	64.81	-0.27
dQwen3.5-9B	6.92B	50B	50.20	52.20	+2.00	58.20	58.20	+0.00
dQwen3.5-9B	6.92B	100B	52.00	51.00	-1.00	56.61	59.79	+3.18

C.5 Full-canvas and block decoding, one token per step

Table 10 gives the complete full-canvas results behind Table 3, while Table 11 repeats the evaluation using 32-token blocks decoded in 32 steps each. The two decoding schemes give very similar accuracy and preserve the model ordering, suggesting that our main comparisons are not particular to full-canvas decoding. Nie et al. [2025b] similarly find that block diffusion sampling benefits their instruction-tuned model but not the base model, which is the setting considered here.

Table 10: **Evaluations for full-canvas decoding at one token per step.** We bold the best DLM number in each column of a size class. Classes with a single size family are left unmarked.

model	trunk	tokens	MMLU	GSM8K	MATH500	HEval	HEval+	MBPP	MBPP+
<i>≈0.5B trunk</i>									
Qwen3.5-0.8B	0.50B	—	50.31	34.95	17.00	22.56	20.73	17.40	24.07
dQwen3.5-0.8B	0.50B	50B	32.62	8.49	5.40	28.05	25.61	22.60	29.10
dQwen3.5-0.8B	0.50B	100B	30.24	8.11	6.20	35.37	31.71	23.20	32.28
<i>≈1.4B trunk</i>									
Qwen3-1.7B	1.41B	—	60.30	74.75	48.40	40.24	37.80	43.80	46.03
Qwen3.5-2B	1.37B	—	57.23	58.98	30.80	37.20	34.15	23.40	31.75
CoDA	1.41B	200B	26.21	2.65	3.80	25.00	20.12	33.80	41.01
dQwen3-1.7B	1.41B	50B	51.30	42.23	13.60	34.76	33.54	30.40	38.89
dQwen3-1.7B	1.41B	100B	48.12	39.35	15.80	41.46	37.20	35.40	46.83
dQwen3.5-2B	1.37B	50B	45.50	20.55	11.00	37.20	32.32	34.20	40.48
dQwen3.5-2B	1.37B	100B	41.55	19.79	9.20	43.90	38.41	33.20	43.12
<i>≈3.6B trunk</i>									
Qwen3.5-4B	3.57B	—	69.82	85.29	49.60	59.15	53.66	40.20	48.41
dQwen3.5-4B	3.57B	50B	62.42	57.77	25.00	54.88	51.83	44.60	51.32
dQwen3.5-4B	3.57B	100B	57.95	53.45	25.60	60.37	57.32	44.60	50.00
<i>6.5–7B trunk</i>									
Qwen2.5-7B	6.53B	—	74.13	79.61	40.80	55.49	48.78	64.40	69.84
Qwen2.5-Coder-7B	6.53B	—	68.07	78.77	37.20	59.76	53.66	68.40	71.43
Qwen3.5-9B	6.92B	—	69.85	87.34	48.80	68.90	62.80	55.00	63.76
LLaDA-8B	6.98B	2.3T	65.88	71.34	28.40	33.54	28.66	40.60	45.50
Dream-7B	6.53B	580B	71.45	74.91	34.80	44.51	38.41	55.40	57.67
Dream-Coder-7B	6.53B	322B	65.41	72.18	29.40	65.85	57.93	62.40	65.08
dQwen3.5-9B	6.92B	50B	74.57	74.98	37.40	64.02	59.15	50.20	58.20
dQwen3.5-9B	6.92B	100B	72.78	69.98	36.80	62.20	58.54	52.00	56.61

Table 11: **Evaluations for block decoding at one token per step.** We bold the best DLM number in each column of a size class. Classes with a single size family are left unmarked.

model	trunk	tokens	MMLU	GSM8K	MATH500	HEval	HEval+	MBPP	MBPP+
<i>≈0.5B trunk</i>									
Qwen3.5-0.8B	0.50B	—	50.31	34.95	17.00	22.56	20.73	17.40	24.07
dQwen3.5-0.8B	0.50B	50B	32.62	8.49	5.20	28.05	25.61	22.20	29.63
dQwen3.5-0.8B	0.50B	100B	30.24	7.88	7.40	34.15	30.49	22.60	32.28
<i>≈1.4B trunk</i>									
Qwen3-1.7B	1.41B	—	60.30	74.75	48.40	40.24	37.80	43.80	46.03
Qwen3.5-2B	1.37B	—	57.23	58.98	30.80	37.20	34.15	23.40	31.75
CoDA	1.41B	200B	26.21	3.18	3.20	23.17	17.07	35.00	42.33
dQwen3-1.7B	1.41B	50B	51.30	41.55	14.40	35.98	33.54	31.20	40.21
dQwen3-1.7B	1.41B	100B	48.12	39.95	15.80	43.29	37.80	36.60	47.35
dQwen3.5-2B	1.37B	50B	45.50	20.92	11.20	39.02	34.15	34.00	40.48
dQwen3.5-2B	1.37B	100B	41.55	19.41	10.40	43.29	37.80	34.40	44.44
<i>≈3.6B trunk</i>									
Qwen3.5-4B	3.57B	—	69.82	85.29	49.60	59.15	53.66	40.20	48.41
dQwen3.5-4B	3.57B	50B	62.42	57.77	26.60	56.10	52.44	46.20	52.65
dQwen3.5-4B	3.57B	100B	57.95	54.66	25.80	59.15	55.49	48.00	51.06
<i>6.5–7B trunk</i>									
Qwen2.5-7B	6.53B	—	74.13	79.61	40.80	55.49	48.78	64.40	69.84
Qwen2.5-Coder-7B	6.53B	—	68.07	78.77	37.20	59.76	53.66	68.40	71.43
Qwen3.5-9B	6.92B	—	69.85	87.34	48.80	68.90	62.80	55.00	63.76
LLaDA-8B	6.98B	2.3T	65.88	72.48	27.20	32.93	28.05	40.40	45.24
Dream-7B	6.53B	580B	71.45	75.89	35.00	43.90	39.02	56.60	58.73
Dream-Coder-7B	6.53B	322B	65.41	72.18	30.20	67.07	58.54	62.00	64.81
dQwen3.5-9B	6.92B	50B	74.57	74.91	38.20	62.20	57.32	51.60	58.47
dQwen3.5-9B	6.92B	100B	72.78	70.58	37.40	62.80	59.15	51.00	58.47

C.6 Additional decode order experiments

We extend the HumanEval decode-order analysis of Section 4.3 across all evaluated DLMs and both full-canvas and block decoding. We use canvas size 256 throughout and block size 16 for block decoding. Table 12 shows that block decoding drives global AR-ness close to 1 by construction, while local AR-ness remains substantially lower, preserving the same local–global distinction observed under full-canvas decoding. The values cited in Section 1 use the full-canvas 256-step setting, and additional trajectories are shown in Figure 10.

Table 12: **Decode order for all evaluated DLMs, over 164 HumanEval problems.** Local (ARL) and global (ARG) AR-ness under each scheme, with NFEs where a threshold sets them. An autoregressive decoder scores 1.000 on both and a random order 0.500.

model	full, 256 steps		full, 128 steps		full, $\tau = 0.9$			block, 256 steps		block, 128 steps		block, $\tau = 0.9$		
	ARL	ARG	ARL	ARG	ARL	ARG	NFE	ARL	ARG	ARL	ARG	ARL	ARG	NFE
LLaDA-8B	0.631	0.932	0.617	0.924	0.630	0.945	83 ± 38	0.679	0.990	0.668	0.991	0.718	0.995	97 ± 39
Dream-7B	0.638	0.884	0.619	0.830	0.659	0.891	139 ± 40	0.706	0.992	0.708	0.993	0.739	0.995	147 ± 39
Dream-Coder-7B	0.652	0.937	0.607	0.934	0.662	0.944	136 ± 55	0.689	0.991	0.680	0.992	0.713	0.994	139 ± 47
CoDA	0.583	0.928	0.593	0.933	0.609	0.952	84 ± 47	0.625	0.987	0.643	0.989	0.701	0.995	94 ± 41
dQwen3-1.7B	0.657	0.974	0.652	0.966	0.679	0.980	145 ± 35	0.695	0.992	0.699	0.993	0.723	0.995	149 ± 33
dQwen3.5-0.8B	0.642	0.974	0.651	0.971	0.651	0.979	144 ± 35	0.681	0.991	0.693	0.993	0.700	0.994	147 ± 32
dQwen3.5-2B	0.646	0.971	0.656	0.970	0.656	0.977	140 ± 41	0.686	0.991	0.703	0.993	0.710	0.995	146 ± 37
dQwen3.5-4B	0.641	0.972	0.644	0.969	0.657	0.978	130 ± 36	0.688	0.991	0.689	0.993	0.716	0.995	135 ± 34
dQwen3.5-9B	0.636	0.967	0.651	0.970	0.655	0.975	129 ± 34	0.691	0.991	0.703	0.993	0.721	0.995	133 ± 34
AR decoder	1.000	1.000	1.000	1.000	1.000	1.000	256	1.000	1.000	1.000	1.000	1.000	1.000	256

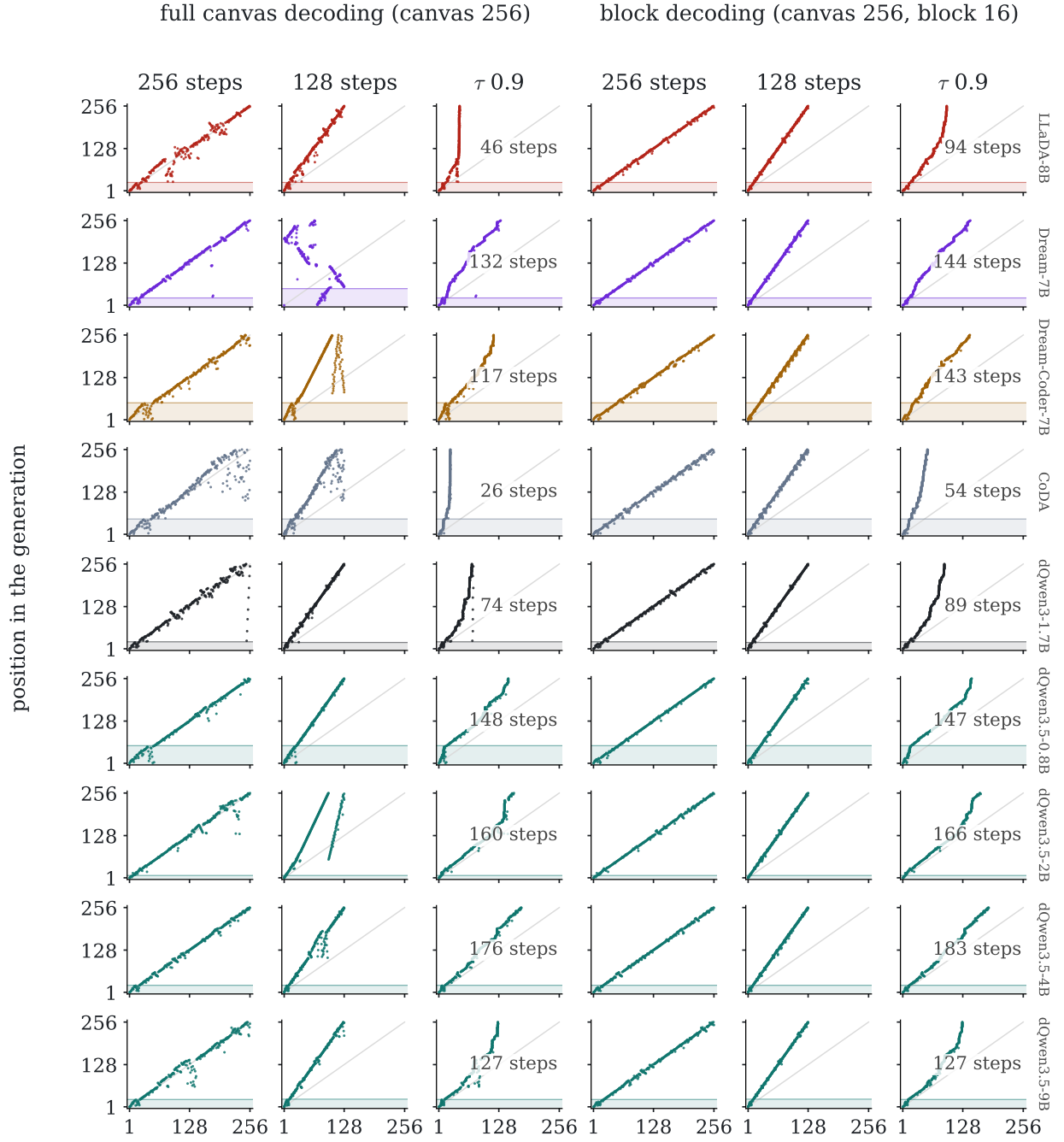
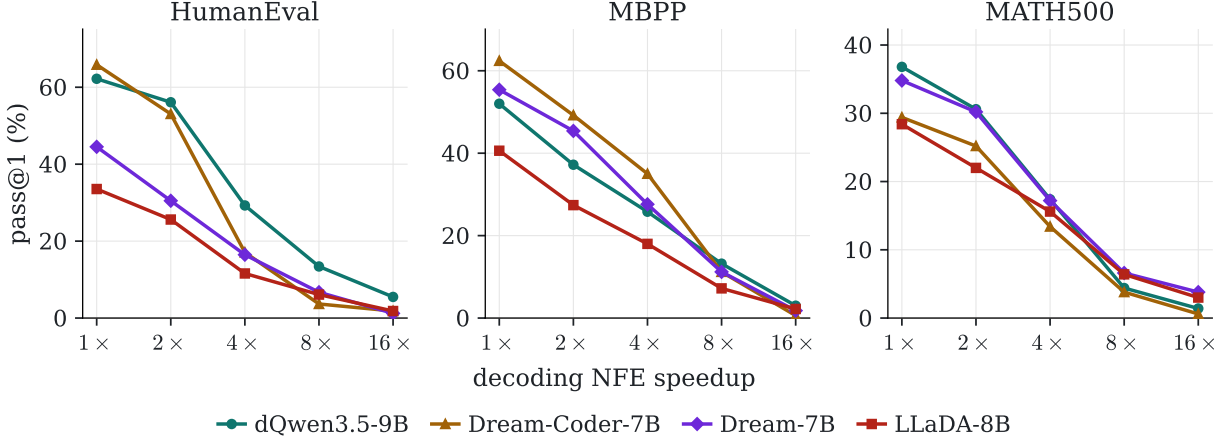


Figure 10: **Decode order on HumanEval1/27, all evaluated DLMs.** Each dot marks when and where a position was unmasked.

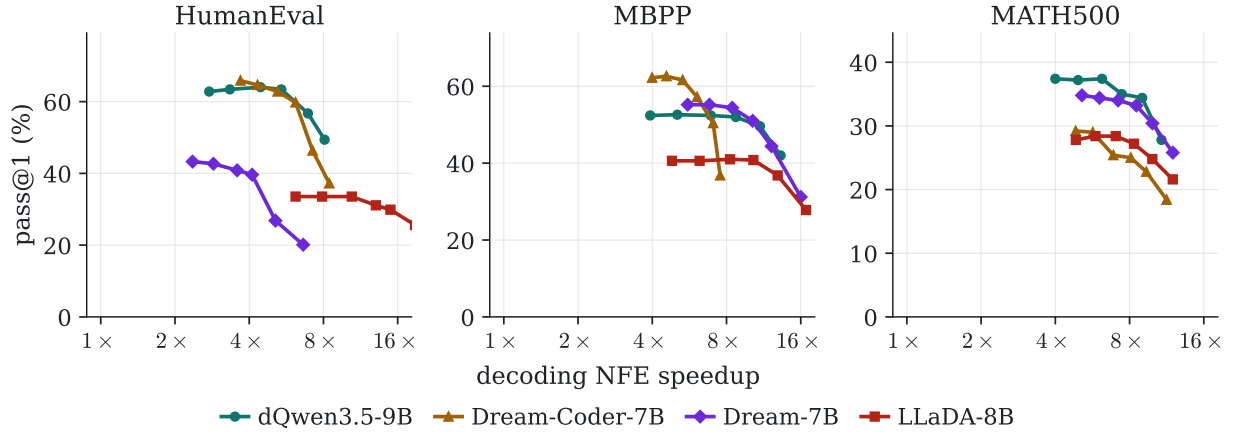
C.7 Additional parallelization experiments

Figures 11 and 12 tabulate the two decoding schemes from Section 4.4, while Figures 13 and 14 extend the comparison to the $\approx 1.4\text{B}$ trunks and Figures 15 and 16 cover the full dQwen3.5 family. We evaluate fixed budgets from 64 to 1024 NFEs and confidence thresholds from $\tau = 0.5$ to $\tau = 0.95$. Across scales, the hybrid models retain meaningful accuracy as the NFE budget falls, with larger models generally supporting more aggressive parallel decoding.



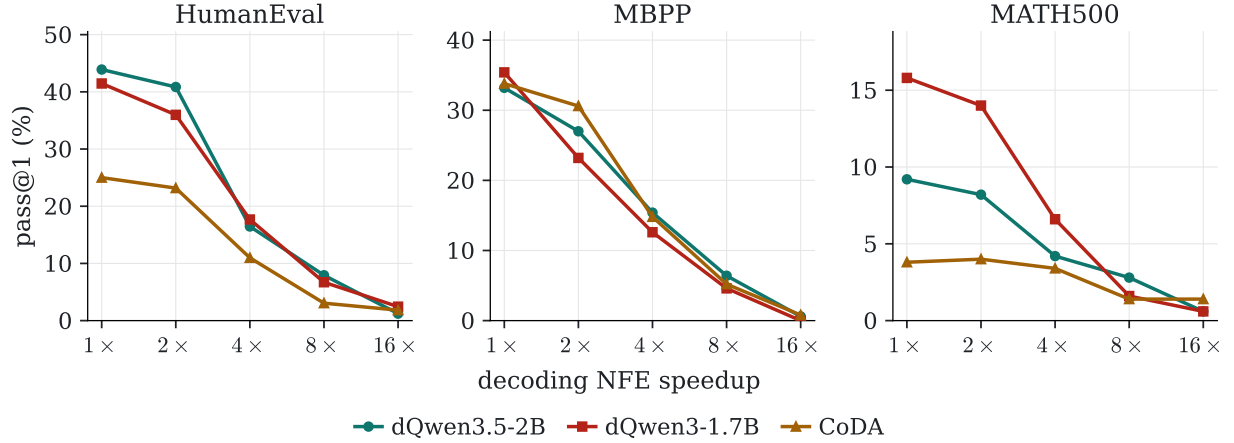
model	HumanEval					MBPP					MATH500				
	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×
dQwen3.5-9B	62.20	56.10	29.27	13.41	5.49	52.00	37.20	25.80	13.20	3.00	36.80	30.60	17.40	4.40	1.40
Dream-Coder-7B	65.85	53.05	17.07	3.66	1.83	62.40	49.20	35.00	11.20	0.60	29.40	25.20	13.40	3.80	0.60
Dream-7B	44.51	30.49	16.46	6.71	1.22	55.40	45.40	27.60	11.20	1.80	34.80	30.20	17.20	6.60	3.80
LLaDA-8B	33.54	25.61	11.59	6.10	1.83	40.60	27.40	18.00	7.20	2.20	28.40	22.00	15.60	6.40	3.00

Figure 11: Parallel decoding at fixed step budgets, 6.5–7B trunks.



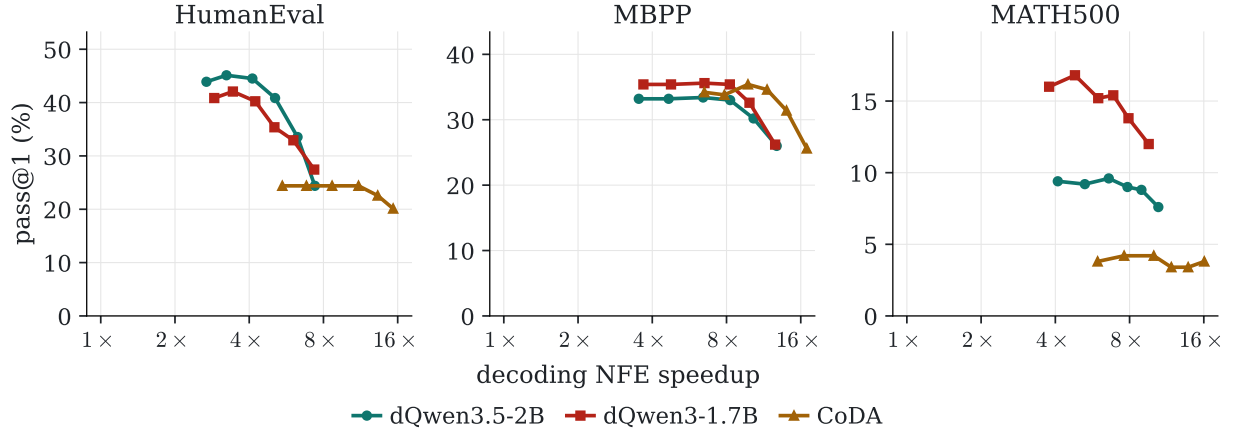
model	HumanEval						MBPP						MATH500					
	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5
dQwen3.5-9B	62.80	63.41	64.02	63.41	56.71	49.39	52.40	52.40	52.00	49.60	42.00	37.40	37.20	37.40	35.00	34.40	27.80	
	2.8×	3.3×	4.4×	5.4×	6.9×	8.1×	3.9×	5.1×	6.9×	8.7×	10.9×	13.3×	4.0×	4.9×	6.2×	7.4×	9.0×	10.8×
Dream-Coder-7B	65.85	64.63	62.80	59.76	46.34	37.20	62.20	62.60	61.60	57.20	50.40	36.80	29.20	29.00	25.40	25.00	22.80	18.40
	3.7×	4.3×	5.2×	6.2×	7.2×	8.4×	4.0×	4.6×	5.3×	6.1×	7.1×	7.5×	4.8×	5.7×	6.9×	8.1×	9.4×	11.3×
Dream-7B	43.29	42.68	40.85	39.63	26.83	20.12	55.20	55.20	54.40	51.00	44.40	31.20	34.80	34.40	34.00	33.20	30.40	25.80
	2.4×	2.9×	3.6×	4.1×	5.1×	6.6×	5.6×	6.8×	8.4×	10.2×	12.2×	16.0×	5.1×	6.0×	7.2×	8.5×	9.9×	12.0×
LLaDA-8B	33.54	33.54	33.54	31.10	29.88	25.61	40.60	40.60	41.00	40.80	36.80	27.80	27.80	28.40	28.40	27.20	24.80	21.60
	6.2×	7.9×	10.4×	13.1×	15.0×	18.8×	4.8×	6.2×	8.3×	10.3×	12.9×	16.8×	4.8×	5.8×	7.0×	8.3×	9.9×	12.0×

Figure 12: Parallel decoding under confidence thresholding, 6.5–7B trunks.



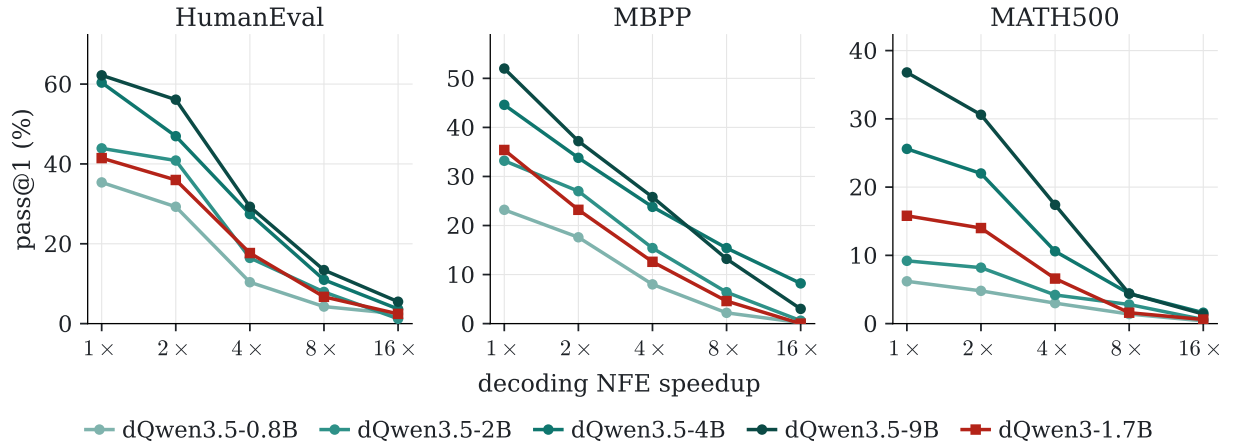
model	HumanEval					MBPP					MATH500				
	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×
dQwen3.5-2B	43.90	40.85	16.46	7.93	1.22	33.20	27.00	15.40	6.40	0.60	9.20	8.20	4.20	2.80	0.60
dQwen3-1.7B	41.46	35.98	17.68	6.71	2.44	35.40	23.20	12.60	4.60	0.00	15.80	14.00	6.60	1.60	0.60
CoDA	25.00	23.17	10.98	3.05	1.83	33.80	30.60	14.80	5.20	0.80	3.80	4.00	3.40	1.40	1.40

Figure 13: Parallel decoding at fixed step budgets, ≈ 1.4 B trunks.



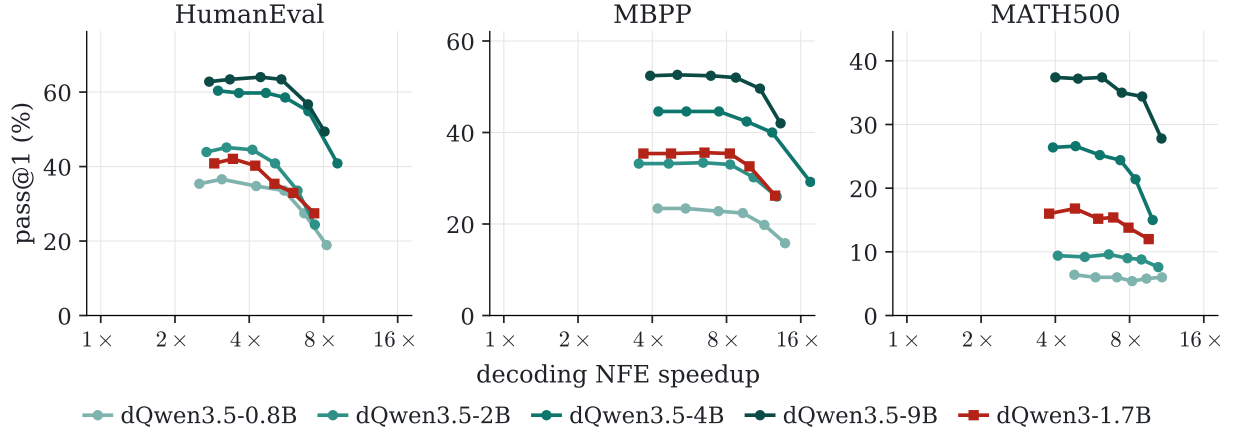
	HumanEval						MBPP						MATH500					
model	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5
dQwen3.5-2B	43.90	45.12	44.51	40.85	33.54	24.39	33.20	33.20	33.40	33.00	30.20	26.00	9.40	9.20	9.60	9.00	8.80	7.60
	2.7×	3.2×	4.1×	5.1×	6.3×	7.4×	3.5×	4.7×	6.4×	8.3×	10.3×	12.8×	4.1×	5.3×	6.6×	7.8×	8.9×	10.5×
dQwen3-1.7B	40.85	42.07	40.24	35.37	32.93	27.44	35.40	35.40	35.60	35.40	32.60	26.20	16.00	16.80	15.20	15.40	13.80	12.00
	2.9×	3.4×	4.2×	5.1×	6.0×	7.3×	3.7×	4.8×	6.5×	8.3×	9.9×	12.6×	3.8×	4.8×	6.0×	6.9×	7.9×	9.6×
CoDA	24.39	24.39	24.39	24.39	22.56	20.12	34.20	33.80	35.40	34.60	31.40	25.60	3.80	4.20	4.20	3.40	3.40	3.80
	5.4×	6.8×	8.7×	11.1×	13.3×	15.3×	6.5×	7.8×	9.8×	11.7×	14.0×	16.9×	5.9×	7.6×	10.0×	11.8×	13.8×	16.1×

Figure 14: Parallel decoding under confidence thresholding, ≈ 1.4 B trunks.



	HumanEval					MBPP					MATH500				
model	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×	1×	2×	4×	8×	16×
dQwen3.5-0.8B	35.37	29.27	10.37	4.27	2.44	23.20	17.60	8.00	2.20	0.20	6.20	4.80	3.00	1.40	0.40
dQwen3.5-2B	43.90	40.85	16.46	7.93	1.22	33.20	27.00	15.40	6.40	0.60	9.20	8.20	4.20	2.80	0.60
dQwen3.5-4B	60.37	46.95	27.44	10.98	3.66	44.60	33.80	23.80	15.40	8.20	25.60	22.00	10.60	4.40	1.60
dQwen3.5-9B	62.20	56.10	29.27	13.41	5.49	52.00	37.20	25.80	13.20	3.00	36.80	30.60	17.40	4.40	1.40
dQwen3-1.7B	41.46	35.98	17.68	6.71	2.44	35.40	23.20	12.60	4.60	0.00	15.80	14.00	6.60	1.60	0.60

Figure 15: Parallel decoding at fixed step budgets, our adapted models.



	HumanEval						MBPP						MATH500					
model	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5	0.95	0.9	0.8	0.7	0.6	0.5
dQwen3.5-0.8B	35.37 2.5x	36.59 3.1x	34.76 4.3x	33.54 5.5x	27.44 6.7x	18.90 8.2x	23.40 4.2x	23.40 5.5x	22.80 7.4x	22.40 9.3x	19.80 11.4x	15.80 13.8x	6.40 4.8x	6.00 5.8x	6.00 7.1x	5.40 8.2x	5.80 9.4x	6.00 10.8x
dQwen3.5-2B	43.90 2.7x	45.12 3.2x	44.51 4.1x	40.85 5.1x	33.54 6.3x	24.39 7.4x	33.20 3.5x	33.20 4.7x	33.40 6.4x	33.00 8.3x	30.20 10.3x	26.00 12.8x	9.40 4.1x	9.20 5.3x	9.60 6.6x	9.00 7.8x	8.80 8.9x	7.60 10.5x
dQwen3.5-4B	60.37 3.0x	59.76 3.6x	59.76 4.7x	58.54 5.6x	54.88 6.9x	40.85 9.1x	44.60 4.2x	44.60 5.5x	44.60 7.5x	42.40 9.7x	40.00 12.3x	29.20 17.5x	26.40 3.9x	26.60 4.8x	25.20 6.1x	24.40 7.3x	21.40 8.5x	15.00 9.9x
dQwen3.5-9B	62.80 2.8x	63.41 3.3x	64.02 4.4x	63.41 5.4x	56.71 6.9x	49.39 8.1x	52.40 3.9x	52.60 5.1x	52.40 6.9x	52.00 8.7x	49.60 10.9x	42.00 13.3x	37.40 4.0x	37.20 4.9x	37.40 6.2x	35.00 7.4x	34.40 9.0x	27.80 10.8x
dQwen3-1.7B	40.85 2.9x	42.07 3.4x	40.24 4.2x	35.37 5.1x	32.93 6.0x	27.44 7.3x	35.40 3.7x	35.40 4.8x	35.60 6.5x	35.40 8.3x	32.60 9.9x	26.20 12.6x	16.00 3.8x	16.80 4.8x	15.20 6.0x	15.40 6.9x	13.80 7.9x	12.00 9.6x

Figure 16: Parallel decoding under confidence thresholding, our adapted models.

C.8 Forward-pass throughput

Because 3/4 of Qwen3.5’s sequence-processing layers are recurrent GDNs, we also ask whether they slow each model forward pass. We measure whole-model throughput across sequence lengths and batch sizes on one NVIDIA GH200 using the standard fast-path kernels: PyTorch flash SDPA for attention and the FLA GDN fast path with `causal_conv1d` for hybrid models (Figure 17). We omit CoDA and Dream-Coder-7B because they share backbone architectures with dQwen3-1.7B and Dream-7B, respectively.

At the $\approx 1.4\text{B}$ trunk scale, dQwen3.5-2B is slower at short sequences but overtakes the full-attention control as sequence length grows. At the 6.5–7B scale, dQwen3.5-9B shows the same trend: it begins slower, but approaches the full-attention models at longer sequences. Thus, causal recurrence does not require slow forward passes; with fast-path kernels, hybrid models remain competitive as sequence length grows while using full attention in only a minority of its layers.

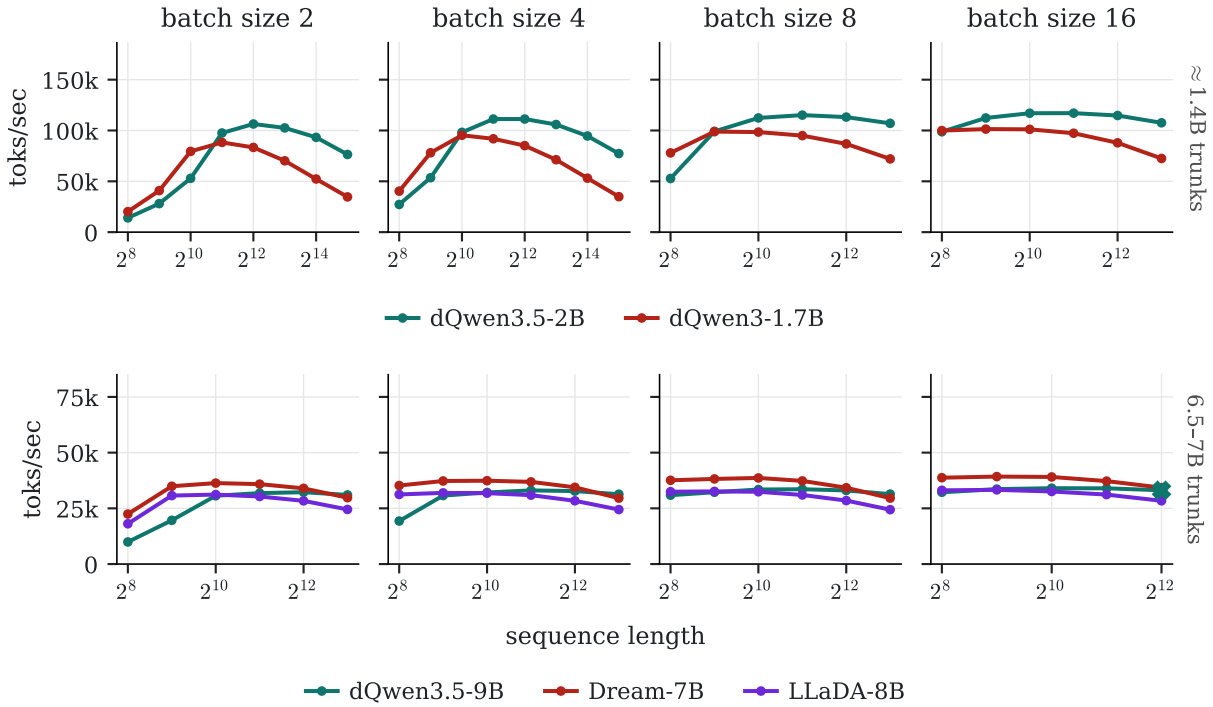


Figure 17: **Forward-pass throughput.** Whole-model throughput on one NVIDIA GH200 using the standard fast-path kernels. At the $\approx 1.4\text{B}$ trunk scale, the hybrid model outperforms its full-attention control at longer sequences despite having more total parameters; at 6.5–7B, it remains competitive with other full-attention DLMs.